程序的链接

为什么学习 编译、链接？

- **帮助构造大型程序**

  未定义的符号、重定义、缺少模块、缺少库 ……

- **避免危险的编程错误**

  定义多个同名的全局变量 ……

- **理解作用域规则是如何实现的**

  语句级、函数级、类级、模块级、全局

  静态（static）变量或函数

- **其他重要的系统概念**

  加载和运行程序、虚拟内存、分页、内存映射

- **利用共享库**

  动态链接库

- **通过解剖文件结构，学习编写程序的技术**

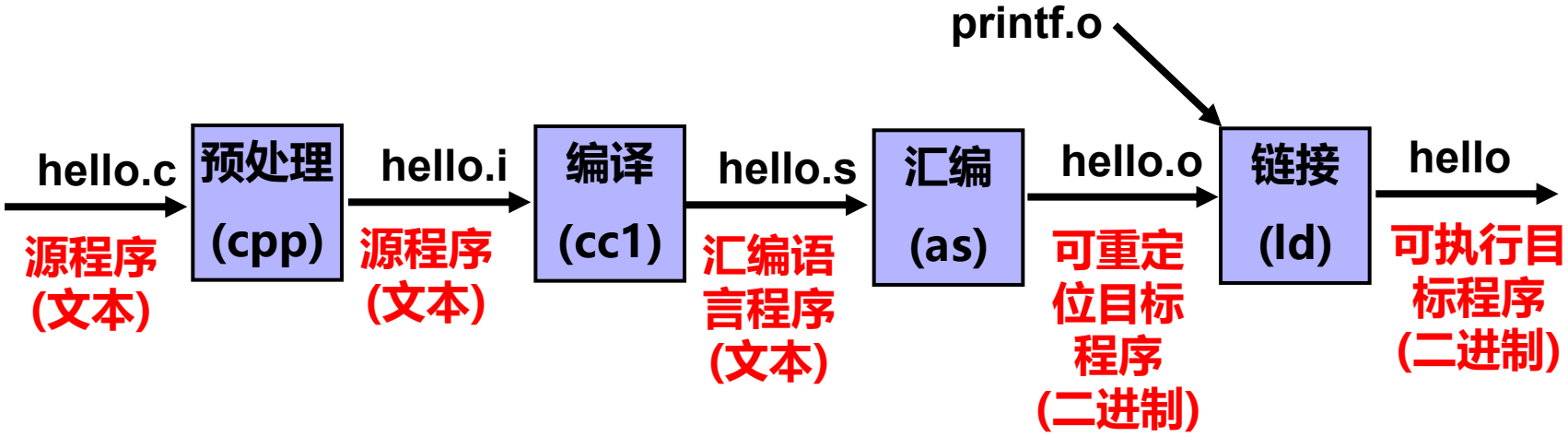
学习内容

- 编译、汇编
- 目标文件格式
- 符号表和符号解析
- 重定位
- 静态链接
- 动态链接

课堂目标

- 能准确描述编译过程，各个阶段所完成的功能；
- 能准确描述可重定位目标文件的结构，包括文件中几个核心节的名字、所存放的信息，这些节之间的关系；
- 对于给定的程序，能准确指出哪些地方需要重定位，以及如何描述需要重定位的信息；
- 对于给定的程序，能准确指出哪些是符号，符号表中存放一些什么信息；
- 能够描述什么是符号解析，什么是重定位；
- 能够描述链接的过程，即生成可执行程序的过程；
- 能够描述静态链接、动态链接的实现机理。

程序编译、汇编



预处理：　gcc -E hello.c -o hello.i

　　或　　cpp hello.c -o hello.i

编　译：　gcc -S hello.c -o hello.s

　　或　　cc -S hello.i -o hello.s

汇　编：　gcc -c -g hello.c -o hello.o　带调试信息

　　或　　as hello.s -o hello.o

反汇编：　objdump -d -S hello.o > result.txt

将结果写到 result.txt 文件中

预处理

预处理 ---- 预编译

从一个 C 程序 到 一个 C程序

编译之前对源代码做某些转换，包括宏替换，头文件替换、不满足编译条件的程序段的删除、注释删除。

```
#define   N   10          // 程序中的N 替换为10
#include  <stdio.h>  //  将stdio.h的内容抄到此处
#ifdef    _windows    //   若未定义符号 _windows
      .....                    //    则不会出现该段代码
#endif
```

预处理

思考：

- 什么是宏定义？ 为什么引入宏定义?
- 头文件中包含什么内容?

  公用数据类型（结构、类）的定义

  全局函数的原型说明、常量符号定义等
- 为什么要使用头文件?
- 什么是条件编译?
- 为什么要条件编译?

预处理

- # 空指令，无任何效果
- #include 包含一个源代码文件
- #define 定义宏
- #undef 取消已定义的宏
- #if 如果给定条件为真，则编译下面代码
- #ifdef 如果宏已经定义，则编译下面代码
- #ifndef 如果宏没有定义，则编译下面代码
- #elif 如果前面的#if给定条件不为真，当前条件为真，则编译下面代码
- #endif 结束一个#if……#else条件编译块

- 预处理指令 是以 # 号开头的代码行
- # 号必须是该行除了任何空白字符外的第一个字符
- # 后是指令关键字，在关键字和 # 号之间允许存在任意个数的空白字符，整行语句构成了一条预处理指令

预处理

#gcc –E test.c –o test.i

预处理后的 文件 为 test.i

```
#include <stdio.h>
#define myfunc(a,b) ((a)>(b)?(a):(b))
#define N 30
int main()
{
   int x=10;
   int y=20;
   int z=15;
   z=N + myfunc(x,y);
   printf("z=%d\n",z);
   return 0;
}
```

```
int main()
{
   int x=10;
   int y=20;
   int z=15;
   z=30 + ((x)>(y)?(x):(y));
   printf("z=%d\n",z);
   return 0;
}
```

提醒：定义宏函数时，注意在参数上加括号

预处理

提醒：定义宏函数时，注意在参数上加括号

```
#define square(x)  x*x

int t;
t = square(1 + 2);
```

t = 1 + 2*1 + 2;

```
#define square(x)  (x)*(x)

t = square(1 + 2);
```

t = (1 + 2)*(1 + 2);

预处理 - 宏的妙用

定义由多条语句组成的宏，简化程序的编写

编写： 对任意函数都能监测其执行时间的程序

```
#define Time_Anyfunc(afuncall)  do {\
    LARGE_INTEGER start, finish, frequency; \
    double duration; \
    QueryPerformanceFrequency(&frequency); \
    QueryPerformanceCounter(&start); \
    afuncall; \
    QueryPerformanceCounter(&finish); \
   duration = (double)(finish.QuadPart - start.QuadPart) * 1000.0 /  \
                  frequency.QuadPart;  \
    printf("Time_Anyfunc : %f 毫秒 \n", duration);  \
 } while(0) ;

Time_Anyfunc(computeAverageScore(s, STUDENTS_NUM) );
```

预处理

条件编译：是什么条件编译？　为什么要写有条件编译的程序？

#gcc –E test.c -D _FIRST –o test.i

```
#include <stdio.h>
int main()
{
   #ifdef _FIRST
        printf("hello , First \n");
   #endif
   #ifdef _SECOND
        printf("good, Second\n");
   #endif
   printf("game over\n");
   return 0;
}
```

```
int main()
{

 printf("hello , First \n");

   printf("game over\n");
   return 0;
}
```

#gcc –E test.c –o test.i

```
int main()
{




   printf("game over\n");
   return 0;
}
```

预处理

#gcc –E -D _SECOND test.c –o test.i

预处理后的 文件 为 test.i

```
#include <stdio.h>
int main()
{
   #ifdef _FIRST
        printf("hello , First \n");
   #endif
   #ifdef _SECOND
        printf("good, Second\n");
   #endif
   printf("game over\n");
   return 0;
}
```

```
int main()
{



 printf("good, Second\n");

   printf("game over\n");
   return 0;
}
```

预处理

#gcc –E -D _SECOND test.c –o test.i

条件编译的选项也可以直接 写在程序中

```
#include <stdio.h>
#define _SECOND
int main()
{
    #ifdef _FIRST
        printf("hello , First \n");
    #endif
    #ifdef _SECOND
        printf("good, Second\n");
    #endif
    printf("game over\n");
    return 0;
}
```

```
int main()
{


 printf("good, Second\n");

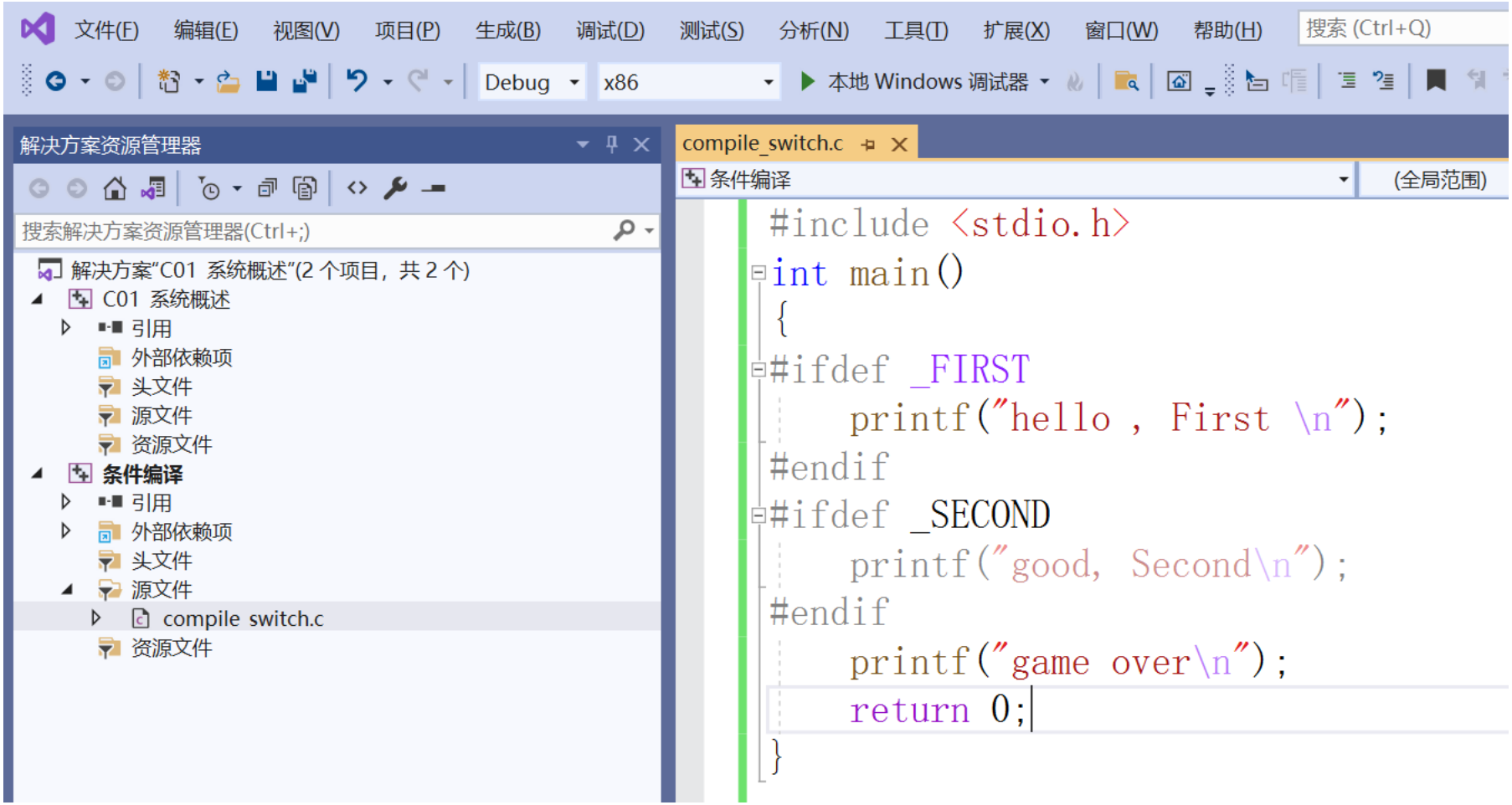
   printf("game over\n");
   return 0;
}
```

#gcc –E test.c –o test.i

#gcc –E -D _FIRST -D _SECOND test.c –o test.i

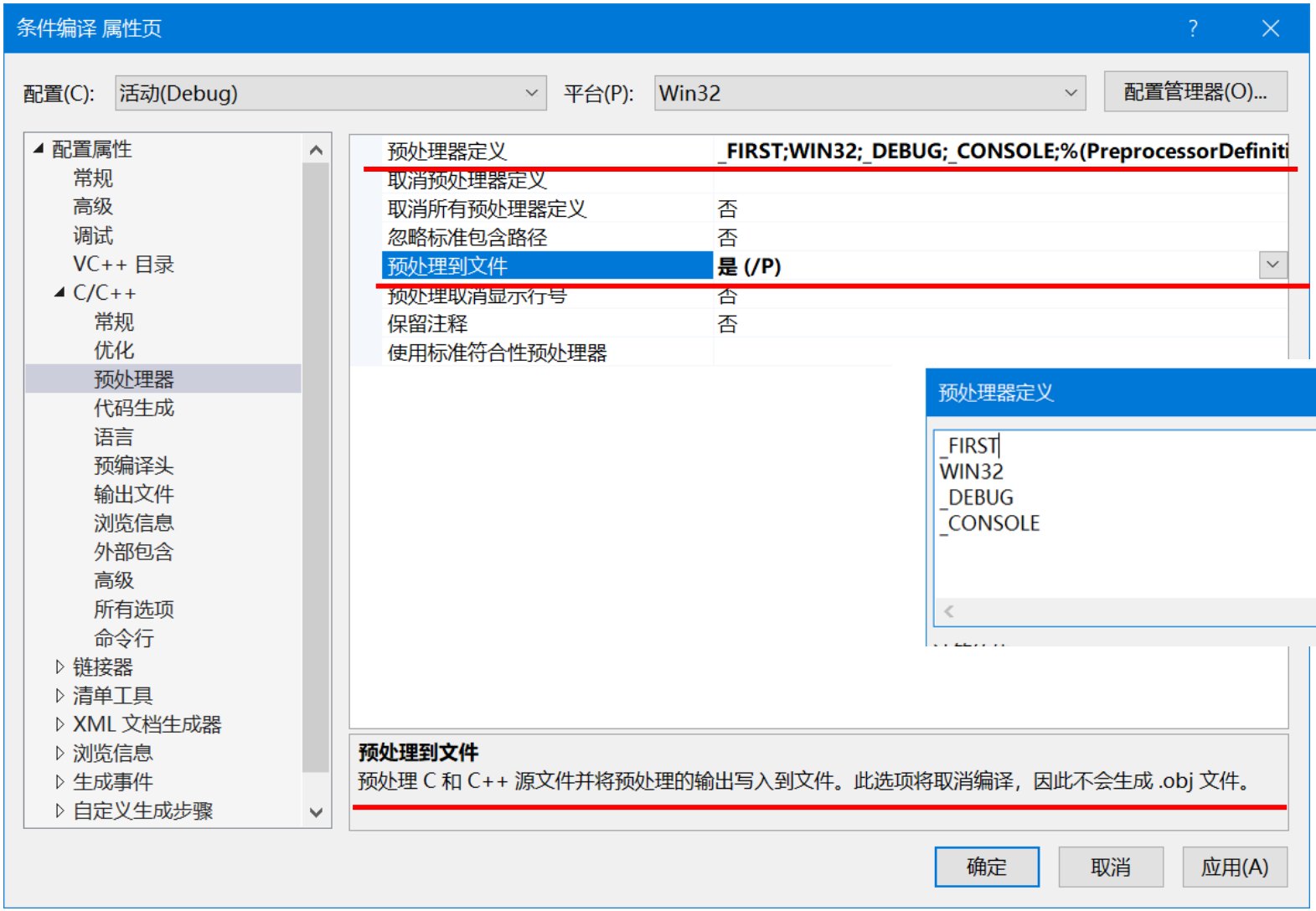
预处理

VS2019 下 C程序编译 预处理



预处理

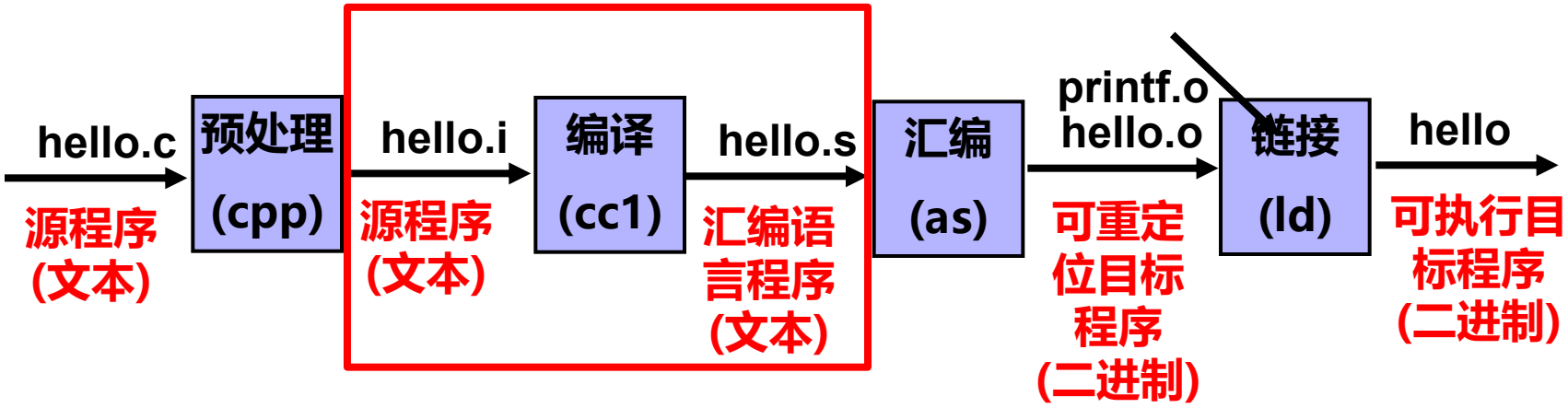
VS2019 下 预处理到文件， 生成 .i 文件



编 译

- 编译　生成汇编语言程序

#gcc -S -D _FIRST test.c –o test.s



编 译

```
 .file   "test.c"
        .text
        .section        .rodata
.LC0:
        .string "hello , First "
.LC1:
        .string "game over"
        .text
        .globl  main
        .type   main, @function
```

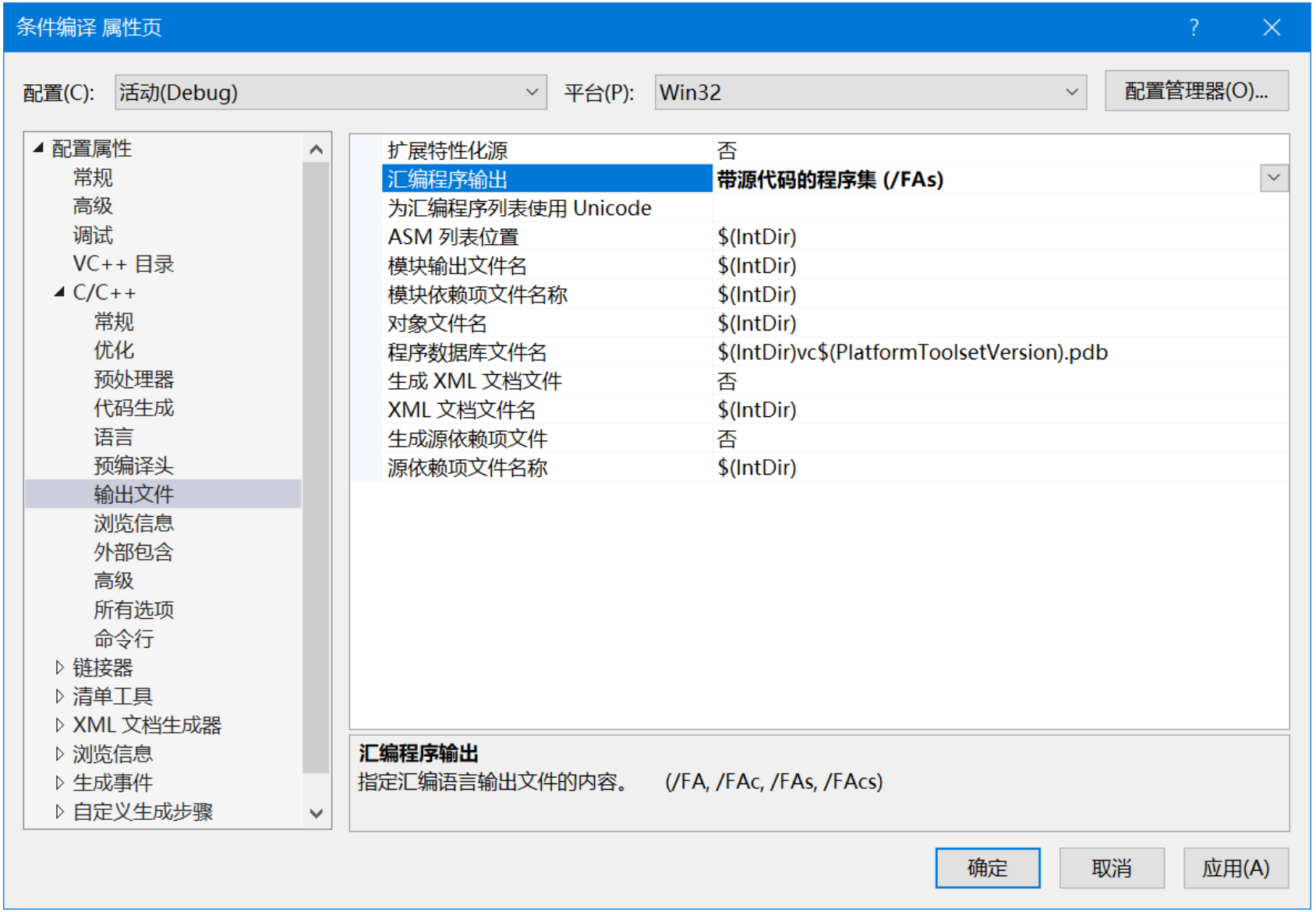
生成的汇编语言程序示例

AT&T 格式

```
main:
.LFB0:
        .cfi_startproc
        pushq   %rbp
        .cfi_def_cfa_offset 16
        .cfi_offset 6, -16
        movq    %rsp, %rbp
        .cfi_def_cfa_register 6
        leaq    .LC0(%rip), %rdi
        call    puts@PLT
        leaq    .LC1(%rip), %rdi
        call    puts@PLT
        movl    $0, %eax
        popq    %rbp
        .cfi_def_cfa 7, 8
        ret
        .cfi_endproc
.LFE0:
        .size   main, .-main
        .ident  "GCC: (Ubuntu ~18.04) 7.5.0"
        .section        .note.GNU-
stack,"",@progbits
```

编 译

- **VS2019 生成汇编语言程序 *.asm 文本文件**



编 译

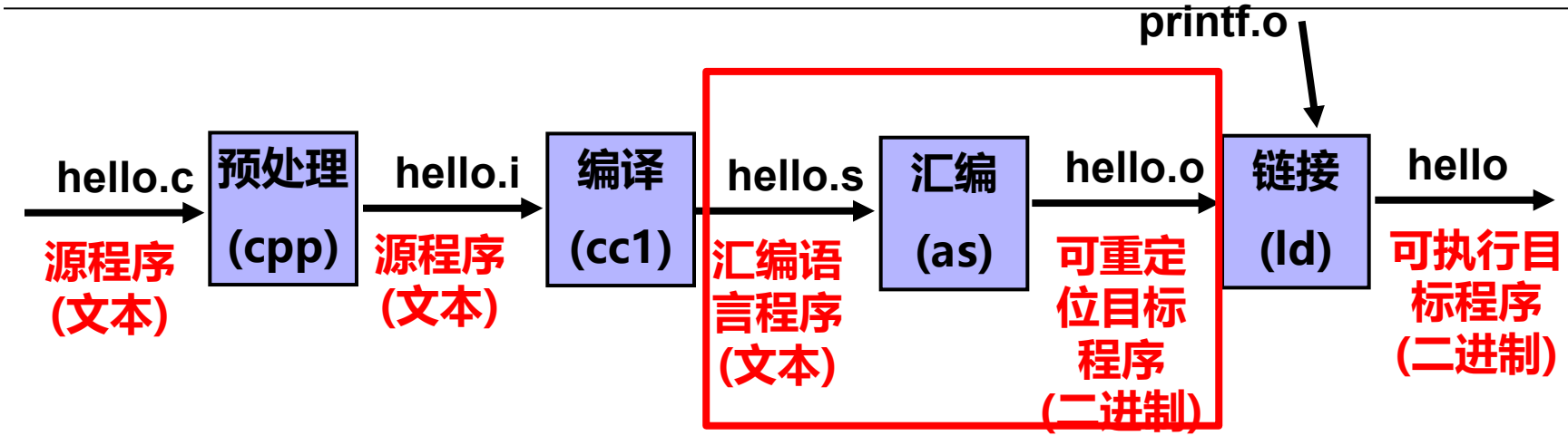
```
_main    PROC                    ; COMDAT
; 3    : {
             push      ebp
             mov       ebp, esp
             sub       esp, 192              ; 000000c0H
             push      ebx
             push      esi
             push      edi
             mov       edi, ebp
             xor       ecx, ecx
             mov       eax, -858993460               ; ccccccccH
             rep stosd
             mov       ecx, OFFSET  …ompile_switch@c
             call      @__CheckForDebuggerJustMyCode@4
; 4    : #ifdef _FIRST
; 5    :     printf("hello , First \n");

             push      OFFSET hello?5?0?5First?5?6@
             call      _printf
             add       esp, 4
; 6    : #endif
```

生成的汇编语言程序示例

Intel 格式

汇 编



- 汇编生成 可重定位目标程序； （文件结构、生成过程）
- 每个文件都是独立汇编的。

  （对外部定义的变量、函数，本文件都要申明）

#gcc -c -D _FIRST test.c –o test.o

#objdump –d test.o 反汇编

#objdump –d –S test.o 带源程序的反汇编

汇 编

#objdump –d test.o

默认用 AT&T

格式显示指令 -M att

```
test.o:     file format elf64-x86-64
Disassembly of section .text:
0000000000000000 <main>:
   0:   55                          push   %rbp
   1:   48 89 e5                    mov    %rsp,%rbp
   4:   48 8d 3d 00 00 00 00        lea    0x0(%rip),%rdi        # b <main+0xb>
   b:   e8 00 00 00 00              callq  10 <main+0x10>
  10:   48 8d 3d 00 00 00 00        lea    0x0(%rip),%rdi        # 17 <main+0x17>
  17:   e8 00 00 00 00              callq  1c <main+0x1c>
  1c:   b8 00 00 00 00              mov    $0x0,%eax
  21:   5d                          pop    %rbp
  22:   c3                          retq
```

```
55                      push   %rbp
48 89 e5                mov    %rsp,%rbp
48 8d 3d ac 0e 00 00    lea    0xeac(%rip),%rdi
e8 f3 fe ff ff          callq  1050 <puts@plt>
48 8d 3d ad 0e 00 00    lea    0xead(%rip),%rdi
e8 e7 fe ff ff          callq  1050 <puts@plt>
b8 00 00 00 00          mov    $0x0,%eax
5d                      pop    %rbp
c3                      retq
```

执行程序的反汇编与.o 文件的反汇编结果，有何差异?

汇 编

#objdump –d -M intel test.o 反汇编

```
test.o:     file format elf64-x86-64
Disassembly of section .text:
0000000000000000 <main>:
   0:   55                          push   rbp
   1:   48 89 e5                    mov    rbp,rsp
   4:   48 8d 3d 00 00 00 00        lea    rdi,[rip+0x0]        # b <main+0xb>
   b:   e8 00 00 00 00              call   10 <main+0x10>
  10:   48 8d 3d 00 00 00 00        lea    rdi,[rip+0x0]        # 17 <main+0x17>
  17:   e8 00 00 00 00              call   1c <main+0x1c>
  1c:   b8 00 00 00 00              mov    eax,0x0
  21:   5d                          pop    rbp
  22:   c3                          ret
```

```
55                      push   rbp
48 89 e5                mov    rbp,rsp
48 8d 3d ac 0e 00 00    lea    rdi,[rip+0xeac]
e8 f3 fe ff ff          call   1050 <puts@plt>
48 8d 3d ad 0e 00 00    lea    rdi,[rip+0xead]
e8 e7 fe ff ff          call   1050 <puts@plt>
b8 00 00 00 00          mov    eax,0x0
5d                      pop    rbp
c3                      ret
```

汇 编



Q：在机器指令中，主要包含哪些信息?

机器指令：操作码、地址码

（寻址方式、地址偏移量、寄存器、立即数等）

Q：在生成机器指令时，哪些信息可以确定，哪些又不能确定?

Q：能够确定地址编码的标识符有哪些?
对应的地址表达形式是什么?

局部变量名（非静态的）→ 地址 n (%rbp) 、n(%rsp)
或者变量与寄存器绑定，不分配空间

参数名 → 地址 n (%rbp) 、n(%rsp)、Register

标号 → 位移量， 与当前指令之间的字节距离

Q：为什么全局变量名、函数名、静态的局部变量，
在汇编时不能确定地址?

汇 编



Q：对于不能确定地址的符号，汇编时怎么处理？

- 在机器码中，先空出要存放的编码空间来

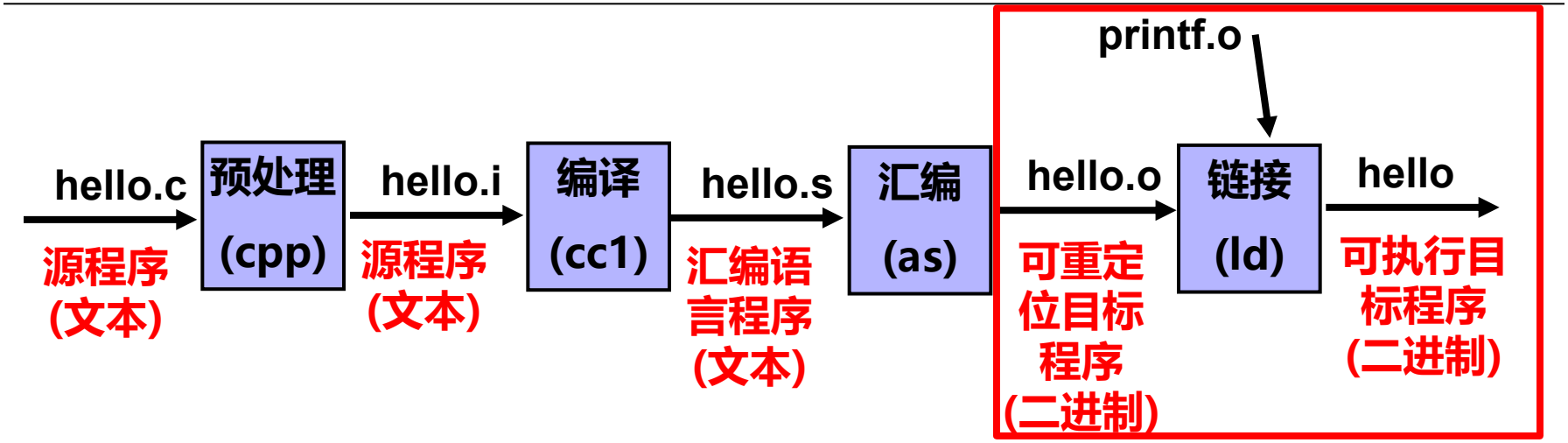
  就像填空，用下划线_____占住空间

  对于 0-1 编码串，可以 先用 00 ...00 占住空间；

- 要记录 在什么位置，需要用什么信息来修改，

  即 重定位信息

链 接



- 链接生成 可执行目标程序； （文件结构、生成过程）
- 将多个模块链接在一起；
- 为符号（全局变量名、函数名、静态的局部变量），给定地址；根据重定位信息和符号的地址，修改机器指令、数据定义等等。
- 对于常量串的地址也进行重定位（将 rodata作为一种特殊符号）

链 接

```
int main()
{
    printf("over\n");
    return 0;
}
```

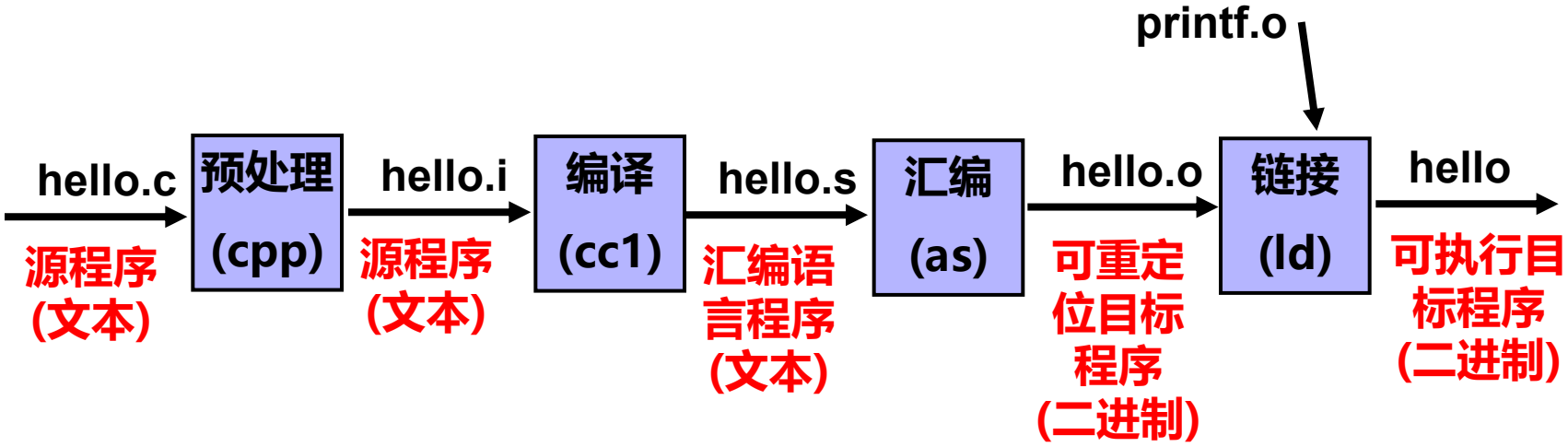
可重定位目标程序

```
 0:   f3 0f 1e fa          endbr64
 4:   55                   push   %rbp
 5:   48 89 e5             mov    %rsp,%rbp
 8:   48 8d 3d 00 00 00 00 lea    0x0(%rip),%rdi
 f:   e8 00 00 00 00       callq  14 <main+0x14>
14:   b8 00 00 00 00       mov    $0x0,%eax
19:   5d                   pop    %rbp
1a:   c3                   retq
```

可执行目标程序

```
1149:       f3 0f 1e fa           endbr64
114d:       55                    push   %rbp
114e:       48 89 e5              mov    %rsp,%rbp
1151:       48 8d 3d ac 0e 00 00  lea    0xeac(%rip),%rdi
1158:       e8 f3 fe ff ff        callq  1050 <puts@plt>
115d:       b8 00 00 00 00        mov    $0x0,%eax
1162:       5d                    pop    %rbp
1163:       c3                    retq
```

小 结



编写源程序后，生成可执行目标程序的过程

目标文件格式

三类目标文件

- 可重定位目标文件 (.o)
  - 其代码和数据可和其他可重定位文件合并为可执行文件
    - 每个.o 文件由对应的.c文件生成
    - 每个.o文件代码和数据地址都从0开始
- 可执行目标文件 (默认为a.out)
  - 包含的代码和数据可以被直接复制到内存并被执行
  - 代码和数据地址为虚拟地址空间中的地址
- 共享的目标文件 (.so)
  - 特殊的可重定位目标文件，称为共享库文件

    能在装入或运行时被装入到内存并自动被链接
  - Windows 中称其为 *Dynamic Link Libraries* (DLLs)

三类目标文件

装入时动态链接（Load-time Dynamic Linking）
运行时动态链接（Run-time Dynamic Linking）

对比项	**装入时链接** (Load-time)	**运行时链接** (Run-time)
链接时机	程序**启动加载时**	程序**运行过程中**
是否必须	必须找到所有 .so，否则程序**直接启动失败**	找不到也不影响程序启动，只是用到时才报错
灵活性	低（编译时就确定依赖）	极高（可随时加载 / 卸载）
内存占用	启动就加载所有依赖库	用到才加载，不用可卸载
使用方式	直接调用函数，编译器自动处理	手动用 dlopen 等 API 加载
典型场景	绝大多数普通程序	插件、驱动、热更新、模块化软件

三类目标文件

可重定位目标文件　VS　共享的目标文件 (.so)

对比项	普通 .o 文件	共享库 .so
阶段	编译中间产物	最终动态库
能否独立加载	❌ 不能	✅ 可以
链接方式	只能**静态链接**	只能**动态链接**
地址模式	纯相对地址，未重定位完	位置无关代码（PIC），可任意加载
内存共享	❌ 不共享	✅ 多个进程共享同一份库代码
更新替换	必须重新编译整个程序	替换 .so 即可，程序不用重新编译
文件大小	小	比 .o 大，带动态链接信息
使用场景	临时中间文件	系统库、第三方库、插件

目标文件的格式

- **目标代码（Object Code）指编译器和汇编器处理源代码后所生成的机器语言目标代码**
- **目标文件（Object File）指包含目标代码的文件**
- **最早的目标文件格式是自有格式，非标准的**
- **几种标准的目标文件格式**
  - **DOS操作系统（最简单）：COM格式，文件中仅包含代码和数据，且被加载到固定位置**
  - **System V UNIX早期版本：COFF格式，包含代码和数据、重定位信息、调试信息、符号表等其他信息，由一组严格定义的数据结构序列组成（Common Object File Format）**
  - **Windows：PE格式（COFF的变种），可移植可执行**
    Portable Executable
  - **Linux等类UNIX：ELF格式（COFF的变种），可执行可链接**
    Executable and Linkable Format

可重定位目标文件的组成成份

代码节　.text;

代码的重定位节　.rela.text

数据节　.data;

数据的重定位节　.rela.data

只读节　.rodata

未初始化数据节　.bss　Block Started by Symbol

符号表节　.symtab

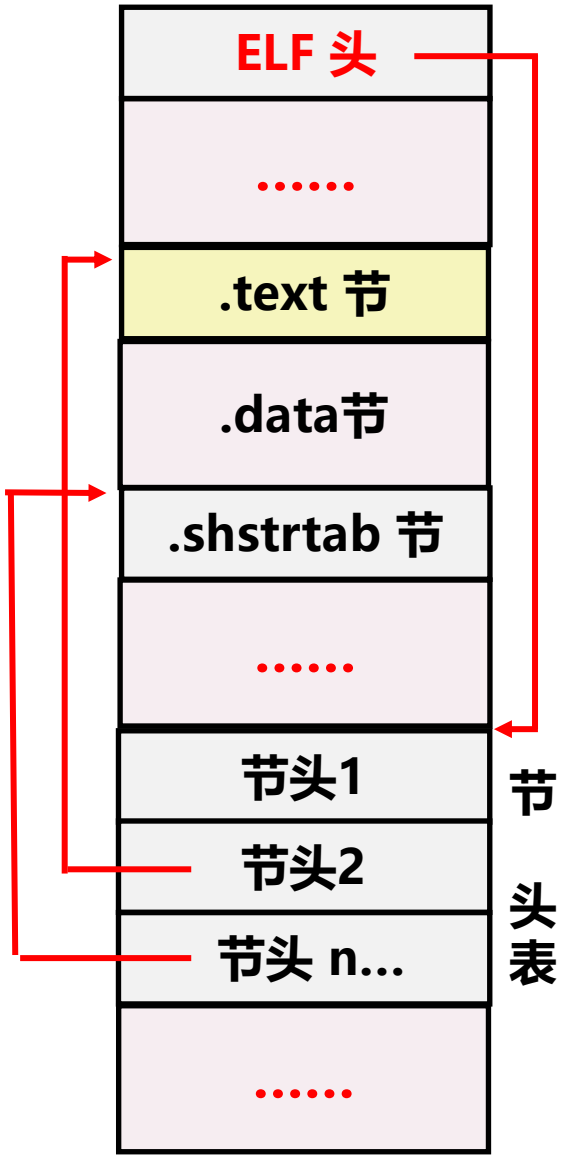
字符串 表节　.strtab

节头表 ：　描述各个节的信息

(节的信息、节在文件中起始位置、节的长度等)

文件头

目标文件格式



ELF头：在文件的开头，固定大小

每个文件的**节数**并不是固定的，有的多，有的少。

节的大小也是不同的，有大有小。

Q：如何设计相应的数据结构?

在ELF头中，有**节头表的起始位置**、节头表中的**节头数目**。

ELF 头结构

ELF 头在目标文件的起始位置

```
#define EI_NIDENT (16)
typedef struct{
    unsigned char  e_ident[EI_NIDENT];  /* Magic number and other info  : 16*/
    Elf64_Half     e_type;              /* Object file type : 2 */
    Elf64_Half     e_machine;           /* Architecture : 2 */
    Elf64_Word     e_version;           /* Object file version : 4 */
    Elf64_Addr     e_entry;             /* Entry point virtual address :8 */
    Elf64_Off      e_phoff;             /* Program header table file offset :8 */
    Elf64_Off      e_shoff;             /* Section header table file offset : 8 */
    Elf64_Word     e_flags;             /* Processor-specific flags : 4 */
    Elf64_Half     e_ehsize;            /* ELF header size in bytes  */
    Elf64_Half     e_phentsize;         /* Program header table entry size */
    Elf64_Half     e_phnum;             /* Program header table entry count */
    Elf64_Half     e_shentsize;         /* Section header table entry size  */
    Elf64_Half     e_shnum;             /* Section header table entry count */
    Elf64_Half     e_shstrndx;          /* Section header string table index */
} Elf64_Ehdr;    // 64 个字节
```

/usr/include/elf.h

ELF头（ELF Header）

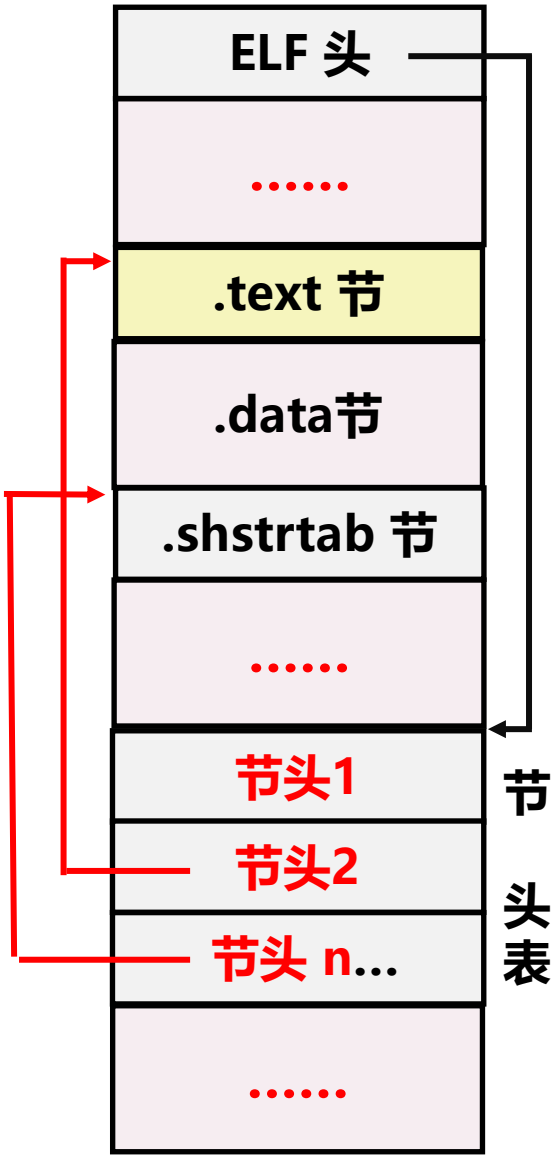
e_ident ： ELF魔数、版本、小端/大端、操作系统平台
e_type ： 目标文件的类型
e_machine ： 机器结构类型
e_version： 目标文件的版本
e_flags ： 处理器标志
e_ehsize： ELF 头的大小
e_entry ： 程序执行的入口地址
e_phoff ： 程序头表（段头表）的起始位置
e_phentsize：程序头表结构的大小（一个表项的长度）
e_phnum ： 程序头表的条目数
e_shoff ： 节头表的起始位置
e_shentsize ： 节头表结构的大小
e_shnum ： 节头表的条目数
e_shstrndx; 节头字符串表的索引

ELF头（ELF Header）

显示ELF 头的信息　　readelf -h test.o

```
root@LAPTOP-CJLSTBTI:/home/chapter4# readelf -h test.o
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF64
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              REL (Relocatable file)
  Machine:                           Advanced Micro Devices X86-64
  Version:                           0x1
  Entry point address:               0x0
  Start of program headers:          0 (bytes into file)
  Start of section headers:          1384 (bytes into file)
  Flags:                             0x0
  Size of this header:               64 (bytes)
  Size of program headers:           0 (bytes)
  Number of program headers:         0
  Size of section headers:           64 (bytes)
  Number of section headers:         19
  Section header string table index: 18
```

节头表



节头表：由多个节头组成

节头

- **对应的节的名字 （名字串的指针）**
- **节在文件中的位置 （位置指针）**
- **节的长度**
- **节的类型 （代码或数据、符号、字符串、无效节 等等）**
- **节标志 （是否可写、可执行 等）**

节头表

```
typedef struct{
    Elf64_Word sh_name;      节名字符串在 .shstrtab中的偏移   4
    Elf64_Word sh_type;      节类型：无效/代码或数据/符号/字符串/... 4
    Elf64_Xword sh_flags;    节标志：该节在虚拟空间中的访问属性  8
    Elf64_Addr  sh_addr;     虚拟地址：若可被加载，则对应虚拟地址  8
    Elf64_Off    sh_offset;  在文件中的偏移地址，对.bss节无意义  8
    Elf64_Xword sh_size;     节在文件中所占的长度  8
    Elf64_Word sh_link;      sh_link和sh_info用于与链接相关的节（如
    Elf64_Word sh_info;      .rel.text节、.rel.data节、.symtab节等）
    Elf64_Xword sh_addralign;    节的对齐要求
    Elf64_Xword sh_entsize;      Entry size if section holds table
} Elf64_Shdr;
```

Executable and Linkable Format

节头描述项 Elf64_Shdr　/usr/include/elf.h

节头表 ： 由若干节头表项组成

sh : section head

节头表

readelf -W -S test.o　　显示各节的描述信息

```
Section Headers:
  [Nr] Name              Type            Address          Off    Size   ES Flg Lk Inf Al
  [ 0]                   NULL            0000000000000000 000000 000000 00      0   0  0
  [ 1] .text             PROGBITS        0000000000000000 000040 00001a 00  AX  0   0  1
  [ 2] .data             PROGBITS        0000000000000000 00005a 000000 00  WA  0   0  1
  [ 3] .bss              NOBITS          0000000000000000 00005a 000000 00  WA  0   0  1
  [ 4] .debug_info       PROGBITS        0000000000000000 00005a 00007b 00      0   0  1
  [ 5] .rela.debug_info  RELA            0000000000000000 0003b8 0000a8 18   I 16   4  8
  [ 6] .debug_abbrev     PROGBITS        0000000000000000 0000d5 000059 00      0   0  1
  [ 7] .debug_aranges    PROGBITS        0000000000000000 00012e 000030 00      0   0  1
  [ 8] .rela.debug_aranges RELA          0000000000000000 000460 000030 18   I 16   7  8
  [ 9] .debug_line       PROGBITS        0000000000000000 00015e 00003b 00      0   0  1
  [10] .rela.debug_line  RELA            0000000000000000 000490 000018 18   I 16   9  8
  [11] .debug_str        PROGBITS        0000000000000000 000199 00005d 01  MS  0   0  1
  [12] .comment          PROGBITS        0000000000000000 0001f6 00002a 01  MS  0   0  1
  [13] .note.GNU-stack   PROGBITS        0000000000000000 000220 000000 00      0   0  1
  [14] .eh_frame         PROGBITS        0000000000000000 000220 000038 00   A  0   0  8
  [15] .rela.eh_frame    RELA            0000000000000000 0004a8 000018 18   I 16  14  8
  [16] .symtab           SYMTAB          0000000000000000 000258 000150 18     17  13  8
  [17] .strtab           STRTAB          0000000000000000 0003a8 00000c 00      0   0  1
  [18] .shstrtab         STRTAB          0000000000000000 0004c0 0000a3 00      0   0  1
Key to Flags:
  W (write), A (alloc), X (execute), M (merge), S (strings), I (info),
  L (link order), O (extra OS processing required), G (group), T (TLS),
  C (compressed), x (unknown), o (OS specific), E (exclude),
  l (large), p (processor specific)
```

节头表 Elf64_Shdr

节名 sh_name: 占 4个字节

节名字符串在.shstrtab中的偏移

.text、.data、.shstrtab……

节类型 sh_type: 4个字节

SHT_NULL	无效节
SHT_PROGBITS	代码或数据节
SHT_SYMTAB	符号表节
SHT_HASH	符号表的哈希表
SHT_STRTAB	字符串节
SHT_RELA	重定位表
SHT_NOBITS	表示该节在文件中没有内容,如.bss节
SHT_DYNAMIC	动态链接信息
SHT_DNYSYM	动态链接的符号表
SHT_NOTE	提示性信息

节头表 Elf64_Shdr

节标志 sh_flags:

该节在虚拟空间中的访问属性

SHF_WRITE (W)	在进程空间中可写
SHF_ALLOC (A)	在进程空间中需要分配空间 包含指示或控制信息的节不分配空间
SHF_EXECINSTR	该节在进程空间中可以被执行
SHF_MERGE	可以被合并
SHF_STRINGS	字符串
SHF_INFO_LINK	与链接相关的节，如重定位表
SHF_LINK_ORDER	

如果节的类型和链接相关，如重定位表、符号表等，
那么sh_link和sh_info两个成员有意义。
对于其他段，这两个成员没有意义。

节头表 示例

```
root@LAPTOP-CJLSTBTI:/home/chapter4# od -Ax -tx1 -v -j0x568 test.o
000568 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
000578 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
000588 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
000598 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0005a8 1b 00 00 00 01 00 00 00 06 00 00 00 00 00 00 00
0005b8 00 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00
0005c8 1a 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0005d8 01 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0005e8 21 00 00 00 01 00 00 00 03 00 00 00 00 00 00 00
0005f8 00 00 00 00 00 00 00 00 5a 00 00 00 00 00 00 00
000608 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
000618 01 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

0x 00000568 处开始节头表；每个节头表项 64个字节

.text节的节头描述

0000001b：节的名字　在节名字符串表（.shstrtab）的位置

00000001 ： 节的类型，1 表示 为 代码或者数据节

0000000000000006 ： 节在虚拟空间中的访问属性

0000000000000000 ： 被加载时对应的虚拟地址

0000000000000040 ： 在文件中的偏移地址

000000000000001a ： 节的长度

节

.text 节

- ✓ 编译汇编后的代码部分

  55 48 89 e5 89 7d ec 89 75 e8 ……

.data 节

- ✓ 已初始化的全局变量、静态局部变量

  int gx=10; int gy=20; ……

  0A 00 00 00 14 00 00 00 ……

.rodata 节 read-only-data

- ✓ 只读数据，如 printf 格式串、switch 跳转表等

  strcpy(dst, “Hello World! “);

.bss 节 Block Started by Symbol

- ✓ 未初始化全局变量、静态局部变量
- ✓ 初始化为0的全局变量、静态局部变量
- ✓ 仅是占位符，不占据任何实际磁盘空间
- ✓ 区分初始化和非初始化是为了节省空间

节

. rela.text 代码节的重定位信息

- ✓ 记录 .text 中重定位的信息，每个需要重定位的位置都对应一条信息

  - 什么位置需要重定位?
  - 用什么符号来定位?
  - 用什么方式来定位?
  - 定位计算的附加项?

. rela.data 数据节的重定位信息

  - 什么位置需要重定位?
  - 用什么符号来定位?
  - 用什么方式来定位?
  - 定位计算的附加项?

节

. symtab　符号表节

- ✓ 只有全局变量、静态的局部变量、函数 才是符号
- ✓ 符号有名字　（在字符串表节中的起始位置）
- ✓ 符号有类型　（数据、函数）
- ✓ 符号有定义、有使用（如使用其他文件定义的符号）
- ✓ 符号在哪一节定义的（代码节、数据节 或者 未定义）？

  外部符号（对于本文件而言）就是未定义的。
  本文件也不知道该外部符号是否真的在其他文件中定义。

- ✓ 每一个符号对应一个描述项
- ✓ 在重定位信息中，指明是符号表的哪一项

. strtab　字符串节

- ✓ 存放各个符号对应的 ASCII 串

小结

代码节　.text;

代码的重定位节　.rela.text

数据节　.data;

数据的重定位节　.rela.data

只读节　.rodata

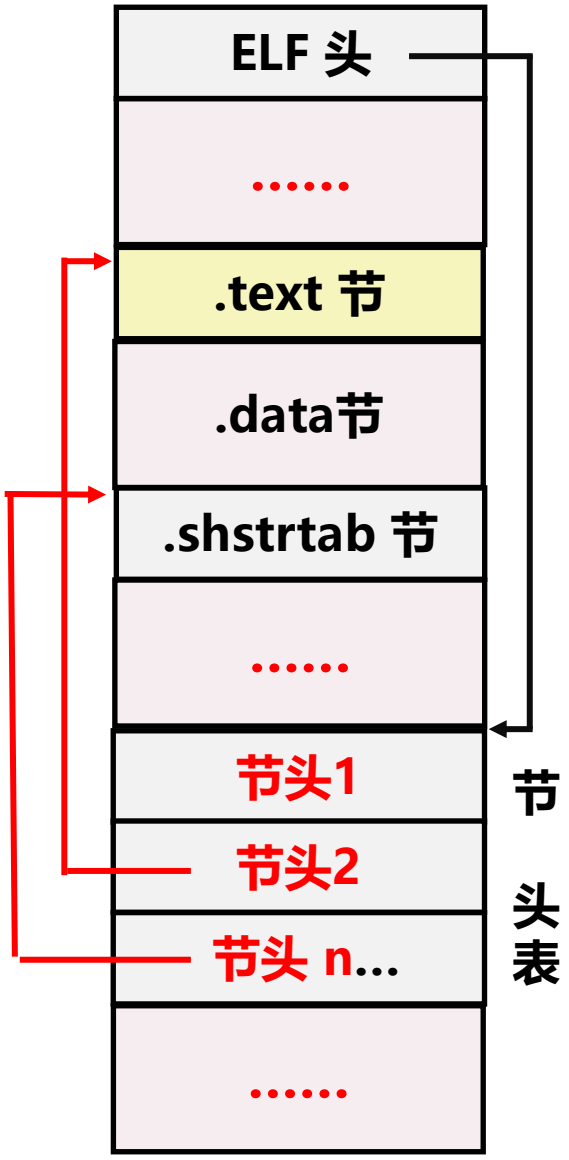
未初始化数据节　.bss　Block Started by Symbol

符号表节　.symtab

字符串 表节　.strtab

节头表 ：　描述各个节的信息　（节的信息、节在文件中起始位置、节的长度等）

文件头



代码节、代码节的重定节

0000000000000000 <add>:　　int g1; 是全局变量　　extern int g2;

```
int add(int i,int j){
      ……
      x+=g1;
15:  8b 05 00 00 00 00    mov    0x0(%rip),%eax
1b:  01 45 fc             add    %eax,-0x4(%rbp)
     x+=g2;
1e:  8b 05 00 00 00 00    mov    0x0(%rip),%eax
 24: 01 45 fc             add    %eax,-0x4(%rbp)
      ……
}
```

.text 节

```
... 8b  05 00 00  00
00 01 45  fc  8b  05
00 00 00  00  01 45
fc ...
```

.rela.text　　代码节的重定位信息

objdump -r test_v.o

```
RELOCATION RECORDS FOR [.text]:
OFFSET           TYPE              VALUE
0000000000000017 R_X86_64_PC32     g1-0x0000000000000004
0000000000000020 R_X86_64_PC32     g2-0x0000000000000004
```

.text节中，地址为 0x17、0x20 处分别要重定位为 g1、g2的地址

-4 是该下一条指令的地址与重定位位置的距离

代码节的重定位信息

直接使用 #readelf -r test.o

```
Relocation section '.rela.text' at offset 0x448 contains 2 entries:
    Offset             Info             Type               Symbol's Value  Symbol's Name + Addend
0000000000000017  0000000d00000002 R_X86_64_PC32          0000000000000000 g1 - 4
0000000000000020  0000000f00000002 R_X86_64_PC32          0000000000000000 g2 - 4
```

```
typedef struct{
    Elf64_Addr     r_offset;   /* Address  要重定位的位置  */
    Elf64_Xword  r_info;      /* Relocation type and symbol index */
                              /* 重定位方式，及符号表的哪一项 */
    Elf64_Sxword      r_addend;     /* Addend 常量加数*/
} Elf64_Rela;     // 24 个字节     24* 2 = 48 = 0x0030
```

每一个**重定位条目对应的结构**： Elf64_Rela
有多少个地方需要重定位，就有多少个条目

代码节的重定位信息

Q: 如何直接在目标文件中以十六进制查看代码节重定位信息？

（1） 确定 .rela.text的起始位置

readelf -S -W test_v.o 【读节头表】

在文件的位置（Off）、 长度（所占字节数,Size）、类型等

```
Section Headers:
  [Nr] Name              Type            Address          Off    Size   ES Flg Lk Inf Al
  [ 0]                   NULL            0000000000000000 000000 000000 00      0   0  0
  [ 1] .text             PROGBITS        0000000000000000 000040 00002c 00  AX  0   0  1
  [ 2] .rela.text        RELA            0000000000000000 000448 000030 18   I 17   1  8
```

(2) 查看 .rela.text 节中文内容

16进制文件的读取 #od -j0x448 -Ax -tx1 test_v.o

```
root@LAPTOP-CJLSTBTI:/home/chapter4# od -j0x448 -Ax -tx1 test_v.o
000448 17 00 00 00 00 00 00 00 02 00 00 00 0d 00 00 00
000458 fc ff ff ff ff ff ff ff 20 00 00 00 00 00 00 00
000468 02 00 00 00 0f 00 00 00 fc ff ff ff ff ff ff ff
```

r_offset ： 0000000000000017

R_info ： 定位方式 00000002 符号编号： 0000000d

r_addend：附加项 ff ff ff ff ff ff ff fc 即 **-4**

代码节的重定位信息

文件内容 与数据结构对应

```
root@LAPTOP-CJLSTBTI:/home/chapter4# od -j0x448 -Ax -tx1 test_v.o
000448 17 00 00 00 00 00 00 00 02 00 00 00 0d 00 00 00
000458 fc ff ff ff ff ff ff ff 20 00 00 00 00 00 00 00
000468 02 00 00 00 0f 00 00 00 fc ff ff ff ff ff ff ff
```

```
typedef struct{
    Elf64_Addr     r_offset;
    Elf64_Xword  r_info;
    Elf64_Sxword   r_addend;
} Elf64_Rela;
```

r_info ： 0x0000000d00000002

符号表的第13项

定位方式 R_X86_64_PC32

#readelf -r test_v.o 以更直接的方式查看重定位信息

```
Relocation section '.rela.text' at offset 0x448 contains 2 entries:
    Offset             Info             Type               Symbol's Value  Symbol's Name + Addend
0000000000000017  0000000d00000002 R_X86_64_PC32          0000000000000000 g1 - 4
0000000000000020  0000000f00000002 R_X86_64_PC32          0000000000000000 g2 - 4
```

readelf -s -W test_v.o 查看符号表

```
    13: 0000000000000000     4 OBJECT  GLOBAL DEFAULT     3 g1
    14: 0000000000000000    44 FUNC    GLOBAL DEFAULT     1 add
    15: 0000000000000000     0 NOTYPE  GLOBAL DEFAULT   UND g2
```

- g1 定义在 第3节(.data)，地址为 0

重定位方式

定位方式： R_X86_64_PC32　　Q：待填的值是怎么计算出来的？

int gx; // 全局变量　#objdump -S -d test.o

要修改的位置

```
 gx=10;
53:   c7 05 00 00 00 00 0a    movl   $0xa,0x0(%rip)
5a:   00 00 00
```

执行取出后，RIP的位置；两者间距为 -8

当前指令的地址　0x000000000800071d;　下一条指令地址　0x08000727

```
 c7 05 e9 08 20 00 0a 00 00 00    movl   $0xa,0x2008e9(%rip)
```

```
(gdb) display &gx
1: &gx = (int *) 0x8201010 <gx>
```

0000000000000055 R_X86_64_PC32　gx-0x0000000000000008

在偏移地址为 0x55处，要重定位： S+A-P

S （该符号的实际地址 0x8201010）+ A （Addend 的值 -8 ） – P（重定位位置的地址 0x800071F） := 0x2008e9

= S - （P-A） = 该符号的实际地址 – 下一条指令的地址(0x8000727)

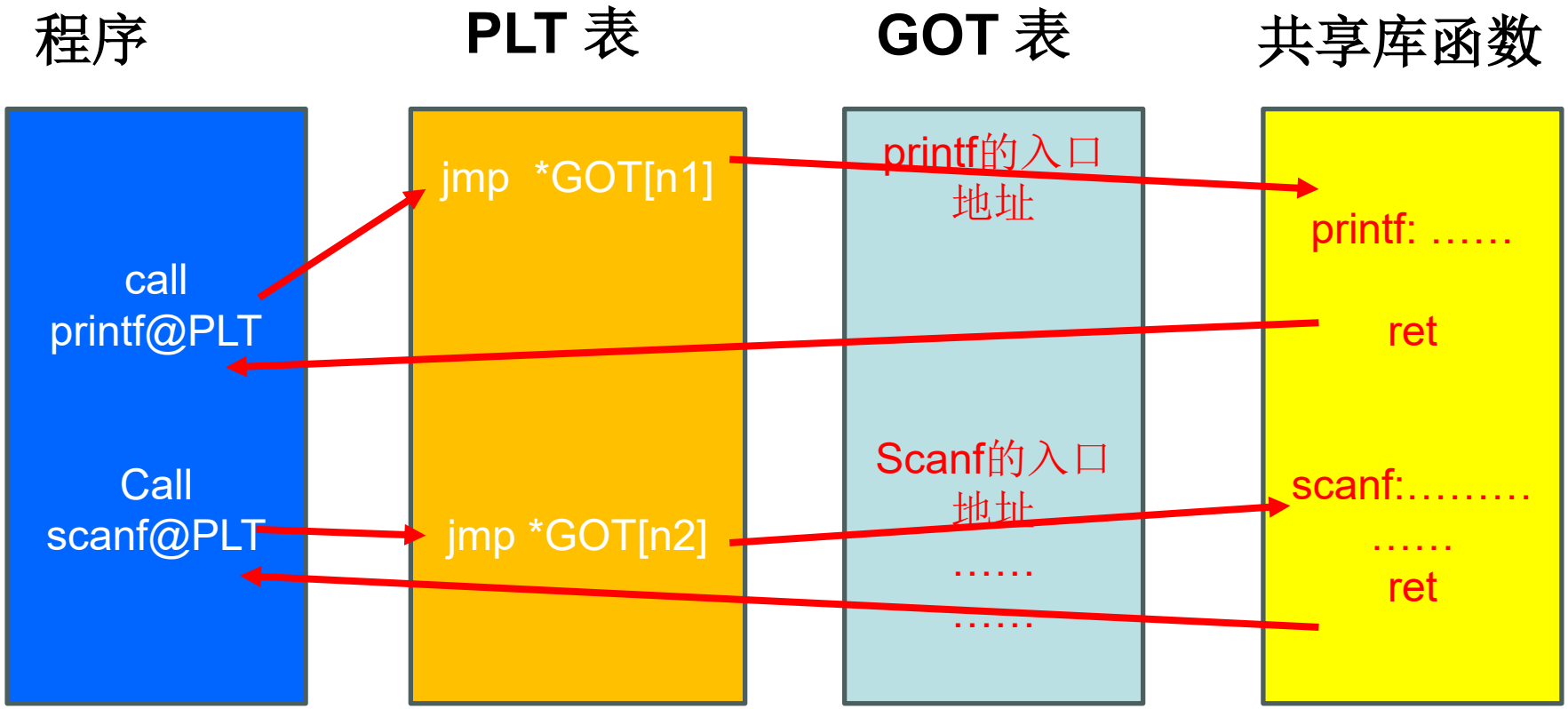
重定位方式

/usr/include/elf.h

```
#define R_X86_64_NONE       0       /* No reloc */
#define R_X86_64_64         1       /* Direct 64 bit  */
#define R_X86_64_PC32       2       /* PC relative 32 bit signed */
#define R_X86_64_GOT32      3       /* 32 bit GOT entry */
#define R_X86_64_PLT32      4       /* 32 bit PLT address */
#define R_X86_64_COPY       5       /* Copy symbol at runtime */
#define R_X86_64_GLOB_DAT   6       /* Create GOT entry */
#define R_X86_64_JUMP_SLOT  7       /* Create PLT entry */
#define R_X86_64_RELATIVE   8       /* Adjust by program base */
#define R_X86_64_GOTPCREL   9 /* 32 bit signed PC relative offset to GOT */
…… ……. …… ……..
```

Q： 程序中，有很多 printf 语句，
每个调用处都涉及重定位问题。
有无简单的实现方式？

重定位方式



Procedure Linkage Table
过程链接表

Global Offset Table
全局偏移表

加载时，只需要填写**GOT**表， 如 第 **n1**表项 为 **printf**的入口地址

Q：若采用普通函数的重定位方式有哪些缺点？
PLT+GOT 有何优点？

重定位方式

PLT： Procedure Linkage Table 过程链接表

是text节的一部分。

GOT：Global Offset Table 全局偏移量表

在数据段的开头处。

在加载程序时，通过动态链接对使用的库中符号进行重定位，而得到GOT，用GOT 记录全局变量的地址、外部函数的地址。而在执行文件中，对于访问的全局变量和函数，只需要给出是GOT 的第几项，从而实现位置无关代码。

.text、.rela.text、.symtab、 .strtab之间的关系

x+=g1; // g1 是全局变量，x 是局部变量

```
15:  8b 05 00 00 00 00    mov    0x0(%rip),%eax
1b:  01 45 fc             add    %eax,-0x4(%rbp)
```

.text

```
00000000  f3 0f 1e fa 55 48 89 e5  89 7d ec 89 75 e8 c7 45
00000010  fc 01 00 00 00 8b 05 00  00 00 00 01 45 fc 8b 05
00000020  00 00 00 00 01 45 fc 8b  45 fc 5d c3 f3 0f 1e fa
```

.rela.text

```
  Offset          Info           Type           Sym. Value    Sym. Name + Addend
000000000017  000900000002 R_X86_64_PC32     0000000000000000 g1 - 4
000000000020  000b00000002 R_X86_64_PC32     0000000000000000 g2 - 4
```

.symtab

```
Symbol table '.symtab' contains 25 entries:
   Num:    Value          Size Type    Bind   Vis      Ndx Name
     9: 0000000000000000     4 OBJECT  GLOBAL DEFAULT    4 g1
```

位置索引

.strtab

```
xu@LAPTOP-CJLSTBTI:~/linkppt$ hexdump -C -n 90 -s 0x6d8 test.o
000006d8  00 74 65 73 74 2e 63 00  67 31 00 61 64 64 00 67  |.test.c.g1.add.g|
000006e8  32 00 6d 61 69 6e 00 70  72 69 6e 74 66 00 00 00  |2.main.printf...|
```

全局变量与静态局部变量

- 在定义全局变量时，已初始化，并在编译时可明确其值

  int gx=10; -> .data 数据节

- 在定义全局变量时，未初始化，或者初始化为 0

  int g_array[3] ; -> .bss 未(或0)初始化数据段

- 在定义全局变量时，已初始化，但初始值编译时不能确定

场景1： int *p = &gx; gx是本模块定义的符号(全局变量)

.data.rel.local .rela.data.rel.local

场景2：int *q = &gy; gy 是其他模块定义的符号(全局变量)

.data.rel .rela.data.rel

涉及 符号表节 .symtab， 字符串节 .strtab

.data节(.data.rel / .data.rel.local) 及 其重定位节

✓已初始化的全局变量、静态局部变量

int gx=10; int gy=20; .....

0A 00 00 00 14 00 00 00 .....

- 在定义全局变量时，可以用其他全局变量或者函数初始化，因而也存在要重定位的问题。

```
int add(int,int);
extern int g2;
int  g1=30;
int  *p = &g1;
int  (*q)(int ,int)=add;
int add(int i,int j)
{   ……
}
```

p 是一个全局变量，其初值需要在链接时才能确定。

编译时会生成一个 .data.rel.local 的节， 将 p 作为该节的一个符号；其初值的重定位信息， 放在 .rela.data.rel.local 节中。

.data.rel.local 节的重定位信息

```
root@LAPTOP-CJLSTBTI:/home/chapter4# readelf -S -W test_rela_data.o
There are 22 section headers, starting at offset 0x788:

Section Headers:
  [Nr] Name              Type            Address          Off    Size   ES Flg Lk Inf Al
  [ 0]                   NULL            0000000000000000 000000 000000 00      0   0  0
  [ 1] .text             PROGBITS        0000000000000000 000040 00002c 00  AX  0   0  1
  [ 2] .rela.text        RELA            0000000000000000 000518 000030 18   I 19   1  8
  [ 3] .data             PROGBITS        0000000000000000 00006c 000004 00  WA  0   0  4
  [ 4] .bss              NOBITS          0000000000000000 000070 000000 00  WA  0   0  1
  [ 5] .data.rel.local   PROGBITS        0000000000000000 000070 000010 00  WA  0   0  8
  [ 6] .rela.data.rel.local RELA            0000000000000000 000548 000030 18   I 19   5  8
  [ 7] .debug_info       PROGBITS        0000000000000000 000080 0000db 00      0   0  1
  [ 8] .rela.debug_info  RELA            0000000000000000 000578 0000f0 18   I 19   7  8
```

.rela.data.rel.local

```
Relocation section '.rela.data.rel.local' at offset 0x548 contains 2 entries:
    Offset             Info             Type               Symbol's Value  Symbol's Name + Addend
0000000000000000  0000000e00000001 R_X86_64_64          0000000000000000 g1 + 0
0000000000000008  0000001100000001 R_X86_64_64          0000000000000000 add + 0
```

需要重定位的位置 （Offset）
用哪个符号来定位 （Info 的前4个字节）
定位方式 （Info 的后4个字节） （ R_X86_64_64 直接的 64位地址）
计算时的附加项 Addend

参见 ： Elf64_Rela

.data.rel.local / .data.rel 节的重定位信息

符号表中的信息

- readelf -s -W test_rela_data.o
- g1 定义在 第3节(.data)，地址为 0

  p，q，定义在 第5节（.data.rel.local），地址分别为 0，8

  add 定义在 第1节 (.text)，地址为 0

  g2 ：未定义

```
Symbol table '.symtab' contains 25 entries:
   Num:    Value          Size Type    Bind   Vis      Ndx Name
    14: 0000000000000000     4 OBJECT  GLOBAL DEFAULT    3 g1
    15: 0000000000000000     8 OBJECT  GLOBAL DEFAULT    5 p
    16: 0000000000000008     8 OBJECT  GLOBAL DEFAULT    5 q
    17: 0000000000000000    44 FUNC    GLOBAL DEFAULT    1 add
    18: 0000000000000000     0 NOTYPE  GLOBAL DEFAULT  UND g2
```

.rodata节 只读数据节

```
int main( )
 {
    char  buf[20];
    printf("hello\n");
    strcpy(buf,"0123456789");
    printf("%s \n",buf);
    return 0;
}
```

Q：printf的第一个参数是格式串的首地址，格式串放在什么地方？

strcpy的第二个串是常量串，编译时不一定还调用strcpy函数，而直接变成一条或多条赋值语句，并不一定放在只读节。

```
# readelf -S -W   rodata.o
```

```
[ 5] .rodata           PROGBITS        0000000000000000 0000b5 00000b
```

```
# hexdump -C –n11 -s 0xb5 rodata.o
```

```
root@LAPTOP-CJLSTBTI:/home/link_ppt# hexdump -C -n11 -s 0xb5 rodata.o
000000b5  68 65 6c 6c 6f 00 25 73  20 0a 00                    |hello.%s ..|
```

.rodata节 的重定位

Q : printf(“%s \n”,buf); 有哪些地方要重定位？

```
# objdump –d -S rodata.o
```

```
42:   48 8d 45 e0            lea    -0x20(%rbp),%rax
46:   48 89 c6                mov    %rax,%rsi
49:   48 8d 3d 00 00 00 00    lea    0x0(%rip),%rdi
50:   b8 00 00 00 00          mov    $0x0,%eax
55:   e8 00 00 00 00          callq  5a <main+0x5a>
```

格式串的首地址要重定位

调用的函数要重定位

```
# readelf   -r  rodata.o
```

```
00000000004c  000500000002 R_X86_64_PC32     0000000000000000 .rodata + 2
000000000056  001200000004 R_X86_64_PLT32    0000000000000000 printf - 4
```

重定位信息

在什么位置要重定位： 004c、0056

用什么符号来重定位： 0005 、0012 0005指的是 rodata

用什么方式进行定位： R_X86_64_PC32、R_X86_64_PLT32

计算的附加项 ： +2， 实际上指的是 rodata的第6个字节

```
root@LAPTOP-CJLSTBTI:/home/link_ppt# hexdump -C -n11 -s 0xb5 rodata.o
000000b5  68 65 6c 6c 6f 00 25 73  20 0a 00                    |hello.%s ..|
```

.rodata节 的只读特性

```
#include <stdio.h>
#include <string.h>
int main()
{  char  buf[20];
  printf("hello\n");
   strcpy(buf,"0123456789");
   printf("%s \n",buf);
   char *p="abcdefghi";
   p[0]='A’ ;
   return 0;
}
```

注意：用 **gcc** 编译 **.c**文件，未给出警告，生成执行程序；但运行时会出错！

只读数据段中的数据是不能修改的！

将文件改成 **.cpp**，用 **g++** 编译，给出了警告，但可以生成执行程序；运行时会出错！

只读数据节

```
root@LAPTOP-CJLSTBTI:/home/link_ppt# hexdump -C -n21 -s0xc7 rodata.o
000000c7  68 65 6c 6c 6f 00 25 73  20 0a 00 61 62 63 64 65  |hello.%s ..abcde|
000000d7  66 67 68 69 00                                    |fghi.|
```

```
char *p="abcdefghi";
5a:   48 8d 05 00 00 00 00    lea    0x0(%rip),%rax
61:   48 89 45 d8             mov    %rax,-0x28(%rbp)
```

代码节

代码节的重定位节，要用 rodata[11] (7+4）处的地址

```
00000000005d  000500000002 R_X86_64_PC32    0000000000000000 .rodata + 7
```

符号表节 .symtab

```
Num:    Value          Size Type    Bind   Vis      Ndx Name
 13: 0000000000000000     4 OBJECT  GLOBAL DEFAULT    3 g1
 14: 0000000000000000    44 FUNC    GLOBAL DEFAULT    1 add
 15: 0000000000000000     0 NOTYPE  GLOBAL DEFAULT  UND g2
```

- g1 定义在 第3节(.data)，value为 0（即其地址为 0）
  （每个节都从地址 0开始，这是暂时的。链接后会变）

  size 为变量（函数）分配的空间大小

  add 定义在 第1节 (.text)，地址为 0，长度 44个字节

  g2 ： 未定义

符号表 解读

```
typedef struct{
  Elf64_Word    st_name;      /* Symbol name (string tbl index) */
  unsigned charst_info;       /* Symbol type and binding */
        // 符号可以是数据、函数. Binding : 本地、全局
  unsigned char   st_other;   /* Symbol visibility */
  Elf64_Section st_shndx;     /* Section index  符号被分配到的节号*/
  Elf64_Addr    st_value;     /* Symbol value : 指的是地址 */
  Elf64_Xword   st_size;      /* Symbol size 符号存储空间的大小*/
} Elf64_Sym;        24个字节
```

```
[17] .symtab             SYMTAB          0000000000000000 0002b0 000180
```

```
 14: 0000000000000000    44 FUNC    GLOBAL DEFAULT    1 add
```

test_v.o 中，有 16个符号条目；16 *24 = 384 = 0x180

对符号 add的描述：　0x2b0 + 0x150 (14*24) = 0x400

名字从字符串表的第13个字符开始 ->可从字符串表取出名字 add;

type 为 2 （st_info的低4位， STT_FUNC =2），

Binding 为 1 （STB_GLOBAL=1）

为 add分配的空间 是 000000000000002c个字节

```
000400 0d 00 00 00 12 00 01 00 00 00 00 00 00 00 00 00
000410 2c 00 00 00 00 00 00 00 11 00 00 00 10 00 00 00
```

字符串节

- 存放各个符号的**ASCII**串
- 名字串之前有 **00**字节作为分隔符（串结束符）

```
# readelf -S -W main.o
```

```
  .strtab           STRTAB          0000000000000000 001140 0000c8
```

```
hexdump  -C  -n 90  -s 0x1140 main.o
```

```
root@LAPTOP-CJLSTBTI:/home/link_ppt# hexdump -C -n90 -s 0x1140 main.o
00001140  00 6d 61 69 6e 2e 63 00  73 74 61 74 69 63 5f 76  |.main.c.static_v|
00001150  2e 32 33 32 34 00 73 74  61 74 69 63 5f 66 75 6e  |.2324.static_fun|
00001160  63 00 73 74 61 74 69 63  5f 76 2e 32 33 33 33 00  |c.static_v.2333.|
00001170  67 69 6e 74 31 00 67 63  68 61 72 00 70 31 00 67  |gint1.gchar.p1.g|
00001180  70 6f 69 6e 74 65 72 00  66 75 6e 63 70 00 65 78  |pointer.funcp.ex|
00001190  74 65 72 6e 5f 66 75 6e  63 00                    |tern_func.|
```

小 结

ELF 头

- ✓ 定义了ELF魔数、版本、小端/大端、操作系统平台、目标文件的类型、机器结构类型、节头表的起始位置和长度等

.text 节

- ✓ 编译汇编后的代码部分

.rodata 节

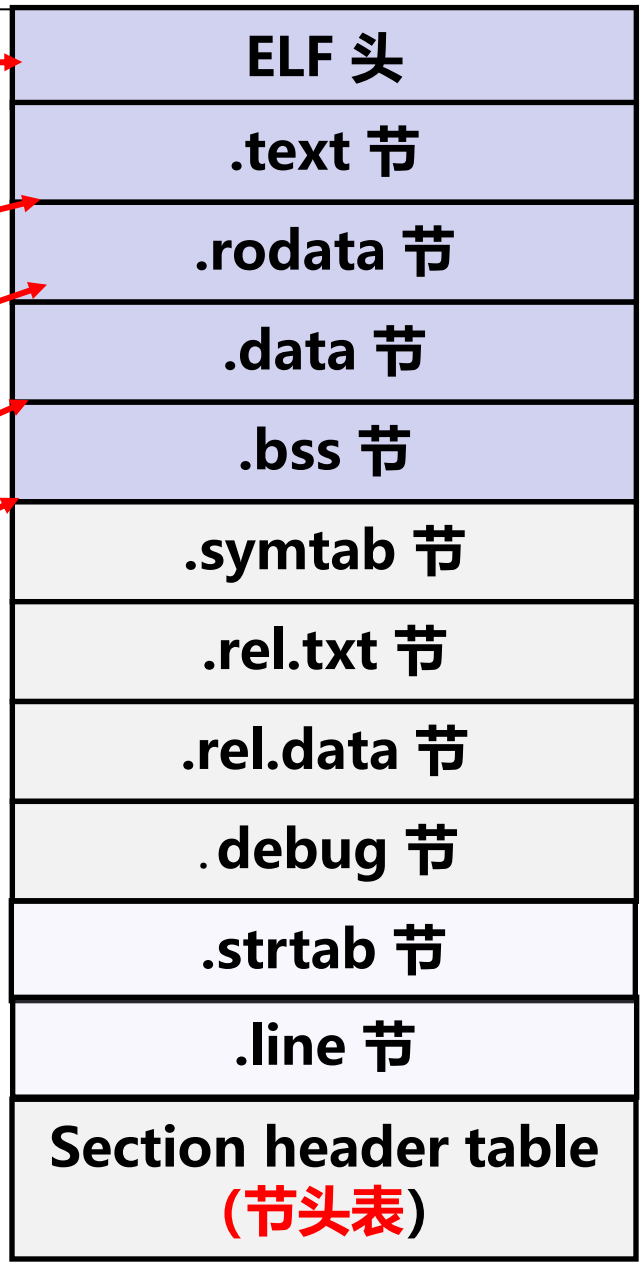
- ✓ 只读数据，如 printf 格式串、switch 跳转表等

.data 节

- ✓ 已初始化的全局变量

.bss 节

- ✓ 未初始化全局变量，仅是占位符，不占据任何实际磁盘空间。区分初始化和非初始化是为了空间效率



测 验

链接

什么是链接?

采用链接方式生成执行程序的优点是什么?

链接的实现方式有哪些?

什么是静态链接? 什么是动态链接?

什么是加载时动态链接? 什么是运行时动态链接?

链接的实现的基本过程是什么?

什么是符号解析?

什么是重定位?

链接

- 使用GCC编译器编译并链接生成可执行程序P:

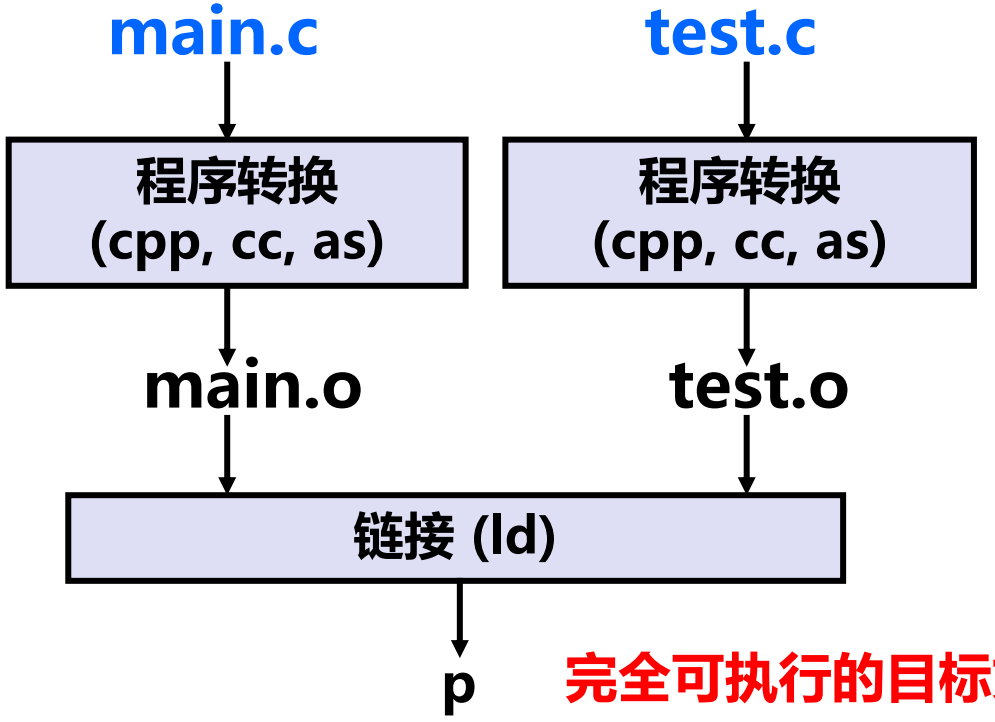
```
# gcc -O2 -g -o p main.c test.c
# ./p
```

-O2：2级优化

-g：生成调试信息

-o：目标文件名

GCC编译器的静态链接过程



源程序文件

分别转换（预处理、编译、汇编）为可重定位目标文件

完全可执行的目标文件

把所有编译后的目标文件、系统库文件整合到一起，解析所有函数、变量地址，打包生成可运行程序。

链接

链接 是 模块化程序设计的必然要求

1）一个程序可以分成很多源程序文件

2）可构建公共函数库，如数学库，标准I/O库等

3）时间上， 各个源程序可分开编译

只需重新编译被修改的源程序文件，然后重新链接

4）空间上，无需包含共享库所有代码

源文件：无需抄共享库函数的源码

可执行文件：无需抄所调用函数的执行代码

运行时内存：只包含所调用函数的代码， 而不是整个共享库

5）简化程序开发

源文件中直接调用共享库函数

链接

静态链接　　**优点？　缺点？**

- 把要调用的函数的执行代码链接到可执行文件中，成为执行文件的一部分。
- 函数的代码就在程序的执行文件中，该文件包含了运行时所需的全部代码。

动态链接

- 所调用的函数代码并没有被拷贝到可执行文件中去
- 仅仅在执行文件中加入了所调用函数的描述信息（往往是一些重定位信息）
- 仅当应用程序被装入内存开始运行时，才在应用程序与相应的共享库之间建立链接关系。

链接

静态链接

动态链接

装载时动态链接(Load-time Dynamic Linking)

编译时明确知道要调用DLL中的哪些函数

运行时动态链接(Run-time Dynamic Linking)

在编译时并不知道将会调用哪些DLL函数，完全是在运行过程中根据需要决定应调用哪个函数，将其加载到内存中；

使用 LoadLibrary和GetProcAddress 类似的函数动态获得DLL函数的入口地址。 dlopen

链接

静态链接

动态链接

装载时动态链接(Load-time Dynamic Linking)

运行时动态链接(Run-time Dynamic Linking)

静态链接库

动态链接库

动态链接与静态链接

#gcc main_link.c –o main_d

#gcc main_link.c /usr/lib/x86_64-linux-gnu/libc.so -o main_dll

C标准库的大部分函数在 libc.so 中，自动链接；

动态链接库以.so作为后缀 （shared object）

对于 非标准库、第三方库等，需要手动添加。

#gcc -static main_link.c -o main_s

静态链接库通常以.a作为后缀

静态链接生成的文件很大

```
-rwxr-xr-x 1 root root   8408 May  7 18:36 main_d
-rwxr-xr-x 1 root root   8408 May  7 23:31 main_dll
-rw-r--r-- 1 root root    147 May  7 18:27 main_link.c
-rwxr-xr-x 1 root root 958808 May  7 18:36 main_s
```

链接第三方库

```
//   math_test.c
#include <stdio.h>
#include <math.h>
#define PI 3.14159265
int main ()
{   double param, result;
    param = 60.0;
    result = cos(param * PI / 180.0 );
    printf ("The cosine of %f   is %f.\n", param, result );
    return 0;
}
```

#gcc math_test.c –o math_test

```
math_test.c:(.text+0x33): undefined reference to  cos'
collect2: error: ld returned 1 exit status
```

#gcc math_test.c –o math_test -lm

数学库：/usr/lib/x86_64-linux-gnu/libm.so / libm.a

链接的实现

链接：将多个可重定位的目标文件合成一个可执行文件。

Q：可重定位的目标文件中包含什么信息？如何合并?

1、符号解析

将符号的引用与一个确定的符号定义建立关联。

符号：全局变量名、函数名、静态的局部变量名

非静态的局部变量名不是符号、参数名不是符号。

2、重定位

在合并生成执行文件时，重新确定每条指令的地址、每个数据的地址、在指令中 确定所引用符号对应的地址。

符号和符号解析

每个可重定位目标模块m都有一个符号表，它包含了在m中定义的符号。有三种链接器符号：

- Global symbols（模块内部定义的全局符号）
  - 由模块m定义并能被其他模块引用的符号。例如，非static 函数和非static的全局变量（指不带static的全局变量）
- External symbols（外部定义的全局符号）
  - 由其他模块定义并被模块m引用的全局符号
- Local symbols（本模块的局部符号）
  - 仅由模块m定义和引用的本地符号。例如，在模块m中定义的带static的函数和全局变量

链接器局部符号不是指程序中的局部变量（分配在栈中的临时性变量）

符号和符号解析

main.c

```
int buf[2] = {1, 2};
extern void swap();

int main()
{
  swap();
  return 0;
}
```

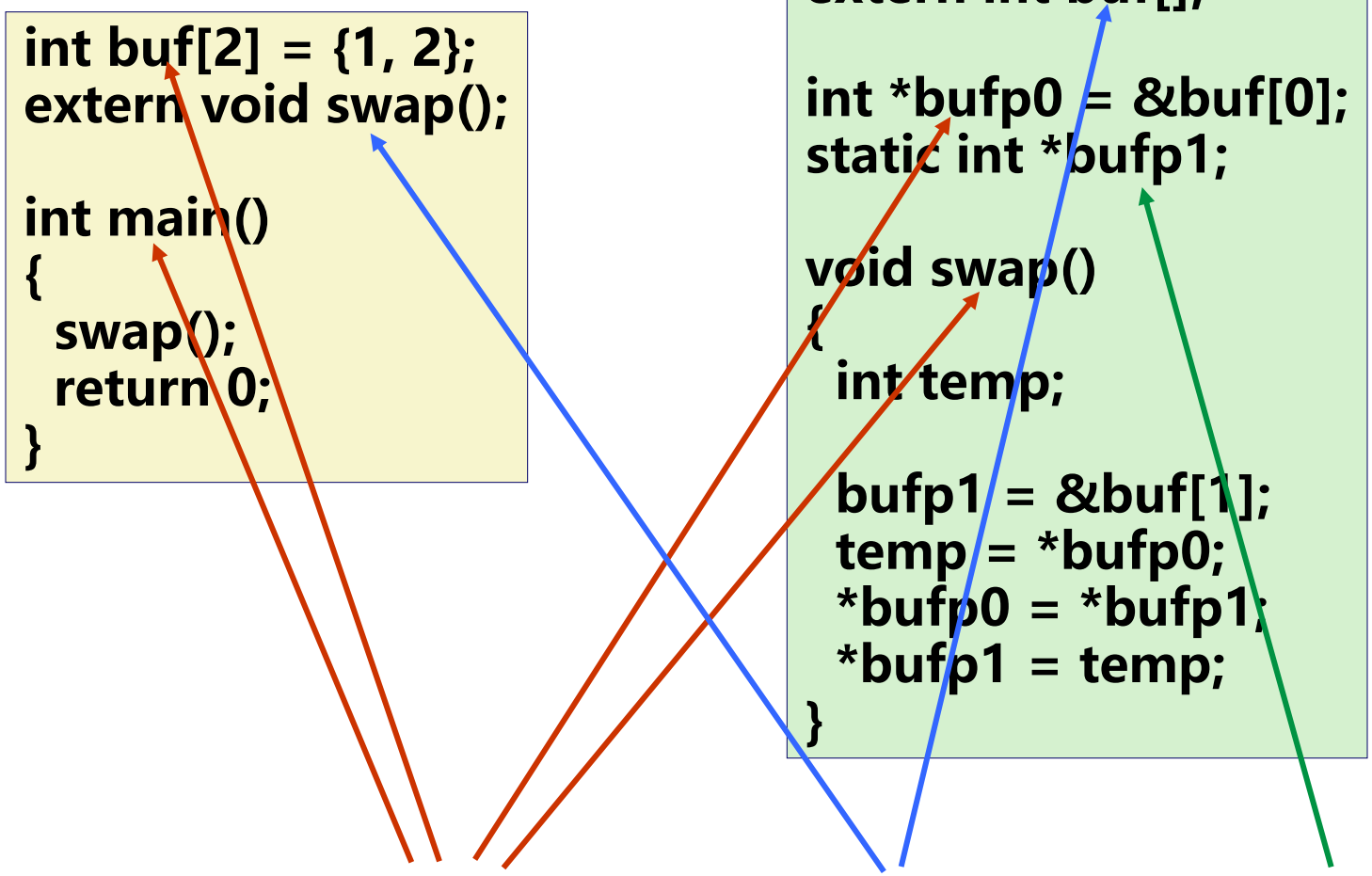
swap.c

```
extern int buf[];

int *bufp0 = &buf[0];
static int *bufp1;

void swap()
{
  int temp;

  bufp1 = &buf[1];
  temp = *bufp0;
  *bufp0 = *bufp1;
  *bufp1 = temp;
}
```

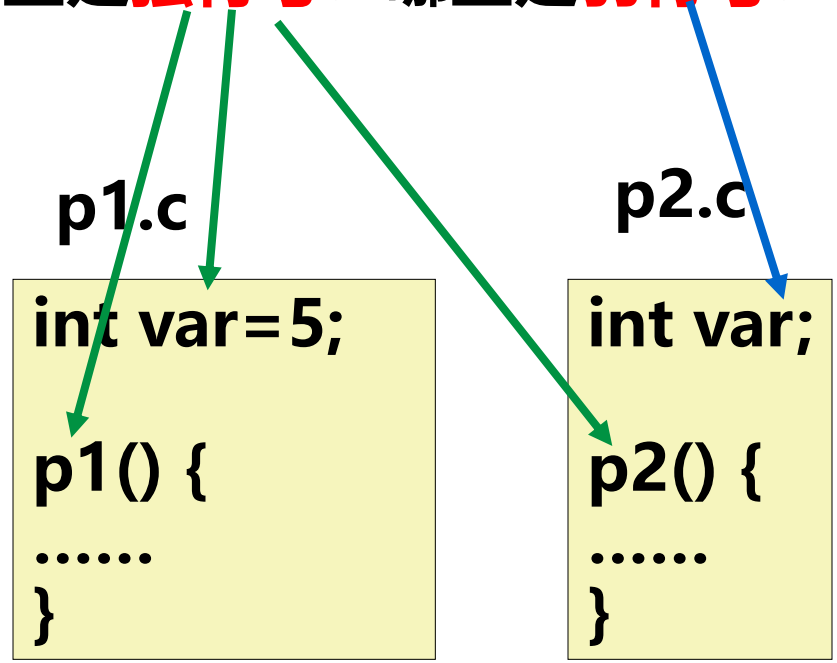


Q: 哪些是全局符号？ 哪些是外部符号？ 哪些是局部符号？

全局符号的符号解析

- **全局符号的强/弱特性　　旧 GCC（GCC 8 及以前）**
  - **函数名和已初始化的全局变量名是强符号**
  - **未初始化的全局变量名是弱符号**

以下符号哪些是强符号？哪些是弱符号？



（GCC 9~）所有全局变量（无论是否初始化）默认为强符号

链接器对符号的解析规则

符号解析时只能有一个确定的定义（即每个符号仅占一处存储空间）

- **多重定义符号的处理规则**

  Rule 1: 强符号不能多次定义

  - **强符号只能被定义一次，否则链接错误**

  Rule 2: 若一个符号被定义为一次强符号和多次弱符号，则按强定义为准

  - **对弱符号的引用被解析为其强定义符号**

  Rule 3: 若有多个弱符号定义，则任选其中一个

  - **使用命令 gcc –fno-common链接时，会告诉链接器在遇到多个弱定义的全局符号时输出一条警告信息。**

多重定义符号的解析举例

以下程序会发生链接出错吗?

```
int  x=10;
int  p1(void);
int main()
{
     x=p1();
     return x;
}
```

main.c

```
int  x=20;
int p1()
{
     return x;
}
```

p1.c

main只有一次强定义

p1 只有一次强定义，

一次申明（不是弱符号定义）

x有两次强定义，所以，链接器将输出一条出错信息

多重定义全局符号的问题

- 尽量避免使用全局变量
- 一定需要用的话，就按以下规则使用
  - 尽量使用本地变量（static）
  - 全局变量要赋初值
  - 外部全局变量要使用extern

多重定义全局变量会造成一些意想不到的错误！

错误默默发生，编译系统不会警告，并会在程序执行很久后才能表现出来，且远离错误引发处。

在一个具有几百个模块的大型软件中，这类错误很难修正。

大部分程序员并不了解链接器如何工作，因而养成良好的编程习惯是非常重要的。

链接器中符号解析的全过程

libc.a无需明显指出！

```
$ gcc –c main.c
$ gcc –static –o myproc main.o ./mylib.a
```

调用关系：main→myfunc1→printf

main.c

```
void myfunc1(viod);
int main()
{
   myfunc1();
   return 0;
}
```

E 将被合并以组成可执行文件的所有目标文件集合
U 当前所有未解析的引用符号的集合
D 当前所有定义符号的集合

开始E、U、D为空，首先扫描main.o，把它加入E，同时把myfun1加入U，main加入D。接着扫描到mylib.a，将U中所有符号（本例中为myfunc1）与mylib.a中所有目标模块（myproc1.o和myproc2.o）依次匹配，发现在myproc1.o中定义了myfunc1，故myproc1.o加入E，myfunc1从U转移到D。在myproc1.o中发现还有未解析符号printf，将其加到U。不断在mylib.a的各模块上进行迭代以匹配U中的符号，直到U、D都不再变化。此时U中只有一个未解析符号printf，而D中有main和myfunc1。因为模块myproc2.o没有被加入E中，因而它被丢弃。

接着，扫描默认的库文件libc.a，发现其目标模块printf.o定义了printf，于是printf也从U移到D，并将printf.o加入E，同时把它定义的所有符号加入D，而所有未解析符号加入U。
处理完libc.a时，U一定是空的，D中符号唯一。

链接器中符号解析的全过程

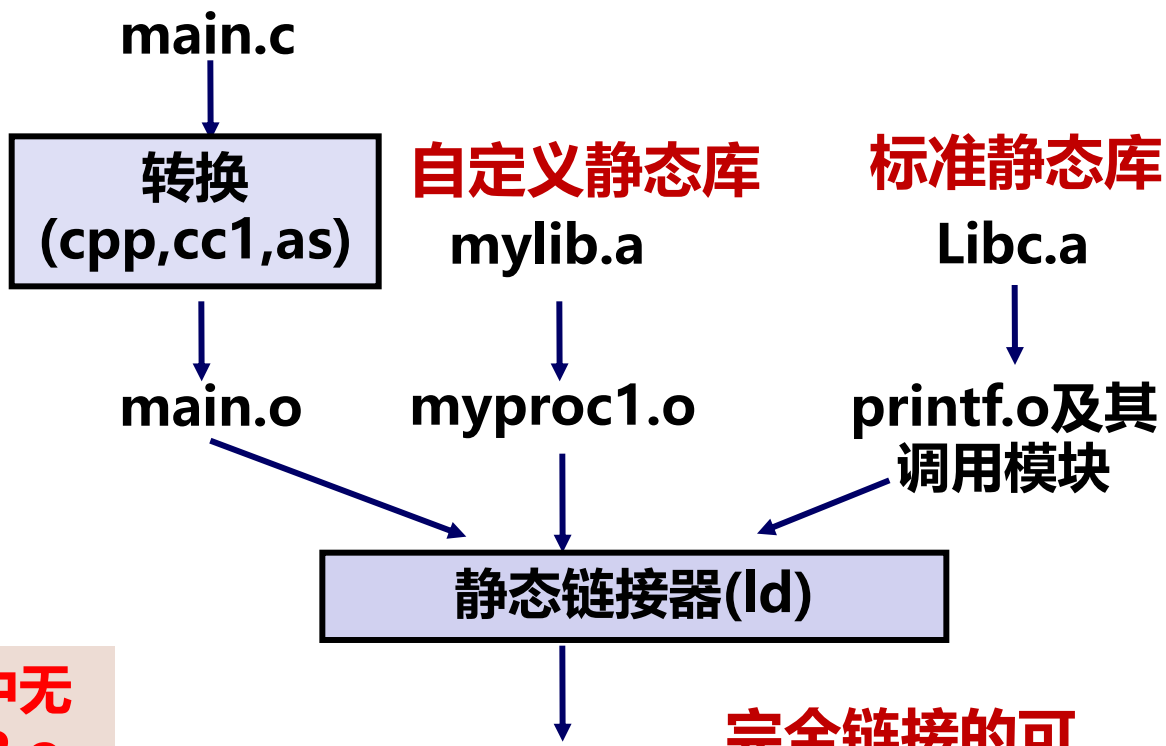
$ ar rcs mylib.a myproc1.o myproc2.o

$ gcc –static –o myproc main.o ./mylib.a

main→myfunc1→printf

main.c

```
void myfunc1(viod);
int main()
{
   myfunc1();
   return 0;
}
```



注意：E中无myproc2.o

解析结果：

E中有main.o、myproc1.o、printf.o及其调用的模块

D中有main、myproc1、printf及其引用的符号

链接器中符号解析的全过程

main.c

```
void myfunc1(viod);
int main()
{
   myfunc1();
   return 0;
}
```

$ gcc –static –o myproc main.o ./mylib.a

解析结果：

E中有main.o、myproc1.o、printf.o及其调用的模块

D中有main、myproc1、printf及其引用符号

main→myfunc1→printf

被链接模块应按调用顺序指定！

若命令为：$ gcc –static –o myproc ./mylib.a main.o， 结果怎样?

首先，扫描mylib，因是静态库，应根据其中是否存在U中未解析符号对应的定义符号来确定哪个.o被加入E。因为U中一开始为空，所以mylib中的myproc1.o和myproc2.o都被丢弃。

然后，扫描main.o，将myfunc1加入U，直到最后它都不能被解析。Why?

因此，出现链接错误！

它只能用mylib.a中符号来解析，而mylib中两个.o模块都已被丢弃！

重定位

符号解析完成后，可进行重定位工作，分三步

- 合并相同的节
  - 将集合E的所有目标模块中相同的节合并成新节

    例如，所有.text节合并作为可执行文件中的.text节
- 对定义符号进行重定位（确定地址）
  - 确定新节中所有定义符号在虚拟地址空间中的地址

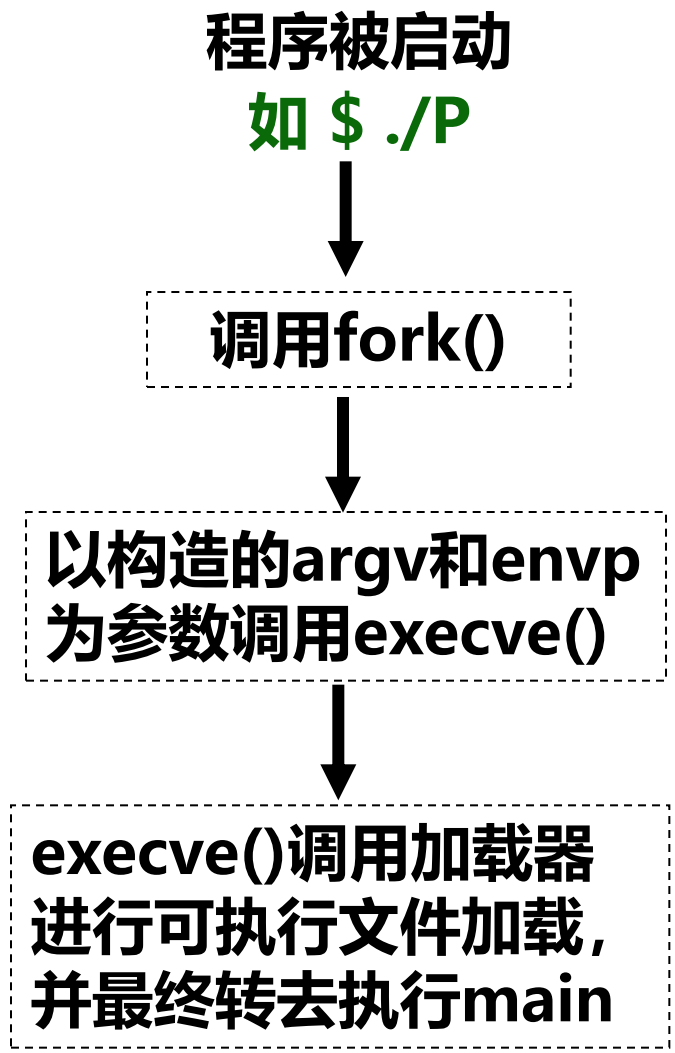
    例如，为函数确定首地址，进而确定每条指令的地址，

    为变量确定首地址
  - 完成这一步后，每条指令和每个全局变量都可确定地址
- 对引用符号进行重定位（确定地址）
  - 修改.text节和.data节中对每个符号的引用（地址）

    需要用到在.rela.data和.rela.text节中保存的重定位信息

可执行文件的加载

- 通过调用execve系统调用函数来调用加载器
- 加载器（loader）根据可执行文件的程序（段）头表中的信息，将可执行文件的代码和数据从磁盘“拷贝”到存储器中（实际上不会真正拷贝，仅建立一种映像，这涉及到许多复杂的过程和一些重要概念，将在后续课上学习）
- 加载后，将PC（EIP）设定指向Entry point (即符号_start处)，最终执行main函数，以启动程序执行。





程序的链接

- 动态链接
  - 动态链接的特性
  - 程序加载时的动态链接
  - 程序运行时的动态链接
  - 动态链接举例

动态链接可以按以下两种方式进行：

- 在第一次加载并运行时进行 (load-time linking).
  - Linux通常由动态链接器(ld-linux.so)自动处理
  - 标准C库 (libc.so) 通常按这种方式动态被链接
- 在已经开始运行后进行(run-time linking).
  - 在Linux中，通过调用 dlopen()等接口来实现
    - 分发软件包、构建高性能Web服务器等

加载时动态链接

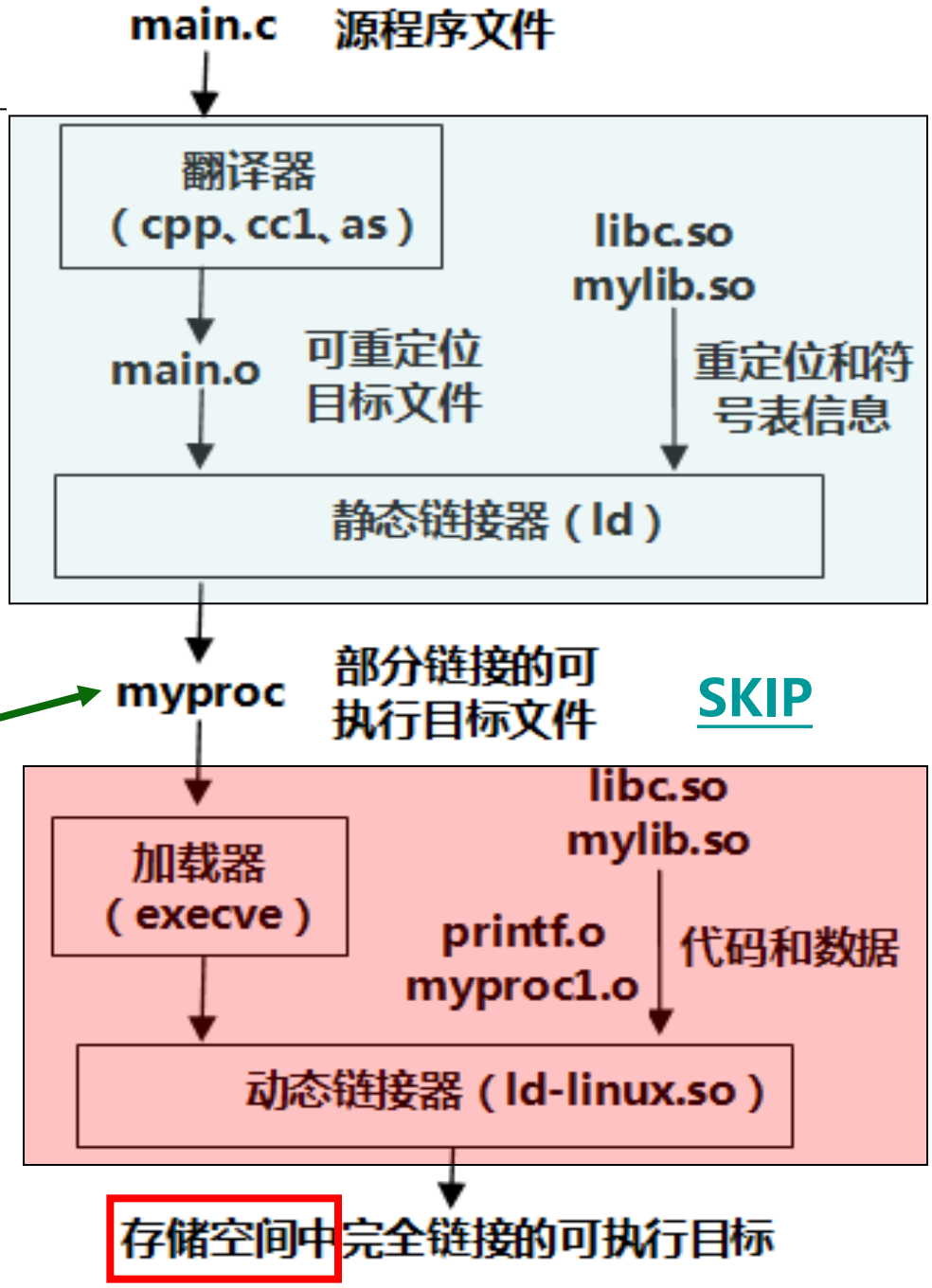
gcc –c main.c　libc.so无需明显指出

gcc –o myproc main.o ./mylib.so

调用关系：main→myfunc1→printf

main.c

```
void myfunc1(viod);
int main()
{
   myfunc1();
   return 0;
}
```



加载 myproc 时，加载器发现在其程序头表中有 .interp 段，其中包含了动态链接器路径名 ld-linux.so，因而加载器根据指定路径加载并启动动态链接器运行。动态链接器完成相应的重定位工作后，再把控制权交给 myproc，启动其第一条指令执行。

加载时动态链接

- 程序头表中有一个特殊的段：INTERP
- 其中记录了动态链接器目录及文件名ld-linux.so

```
Program Headers:
  Type           Offset   VirtAddr   PhysAddr   FileSiz MemSiz  Flg Align
  PHDR           0x000034 0x08048034 0x08048034 0x00100 0x00100 R E 0x4
  INTERP         0x000134 0x08048134 0x08048134 0x00013 0x00013 R   0x1
      [Requesting program interpreter: /lib/ld-linux.so.2]
  LOAD           0x000000 0x08048000 0x08048000 0x004d4 0x004d4 R E 0x1000
  LOAD           0x000f0c 0x08049f0c 0x08049f0c 0x00108 0x00110 RW  0x1000
  DYNAMIC        0x000f20 0x08049f20 0x08049f20 0x000d0 0x000d0 RW  0x4
  NOTE           0x000148 0x08048148 0x08048148 0x00044 0x00044 R   0x4
  GNU_STACK      0x000000 0x00000000 0x00000000 0x00000 0x00000 RW  0x4
  GNU_RELRO      0x000f0c 0x08049f0c 0x08049f0c 0x000f4 0x000f4 R   0x1
```

运行时动态链接

调用**动态链接器接口**提供的函数

在运行时进行动态链接

dlopen

dlsym

dlerror

dlclose ……

其头文件为dlfcn.h

```
#include <stdio.h>
#include <dlfcn.h>
int main()
{   void *handle;
    void (*myfunc1)();
    char *error;
     /* 动态装入包含函数myfunc1()的共享库文件 */
    handle = dlopen("./mylib.so", RTLD_LAZY);
    if (!handle) {
            fprintf(stderr, "%s\n", dlerror());
            exit(1);
     }
     /* 获得一个指向函数myfunc1()的指针myfunc1*/
     myfunc1 = dlsym(handle, "myfunc1");
     if ((error = dlerror()) != NULL) {
            fprintf(stderr, "%s\n", error); exit(1);  }
     /* 现在可以像调用其他函数一样调用函数myfunc1() */
     myfunc1();
     /* 关闭（卸载）共享库文件 */
     if (dlclose(handle) < 0) {
            fprintf(stderr, "%s\n", dlerror());
            exit(1);
     }
     return 0;
}
```

位置无关代码（PIC）

- 动态链接用到一个重要概念：
  - 位置无关代码（Position-Independent Code，PIC）
  - GCC选项-fPIC指示生成PIC代码
- 共享库代码是一种PIC
  - 共享库代码的位置可以是不确定的
  - 即使共享库代码的长度发生变化，也不影响调用它的程序
- 引入PIC的目的
  - 链接器无需修改代码即可将共享库加载到任意地址运行
- 所有引用情况

  (1) 模块内的过程调用、跳转，采用PC相对偏移寻址

  (2) 模块内数据访问，如模块内的全局变量和静态变量

  (3) 模块外的过程调用、跳转

  (4) 模块外的数据访问，如外部变量的访问

要实现动态链接，必须生成PIC代码

要生成PIC代码，主要解决这两个问题

位置无关代码（PIC）

myproc1.c

```
# include <stdio.h>
void myfunc1()
{
    printf("%s","This is myfunc1!\n");
}
```

myproc2.c

```
# include <stdio.h>
void myfunc2()
{
    printf("%s","This is myfunc2\n");
}
```

PIC：Position Independent Code

位置无关代码

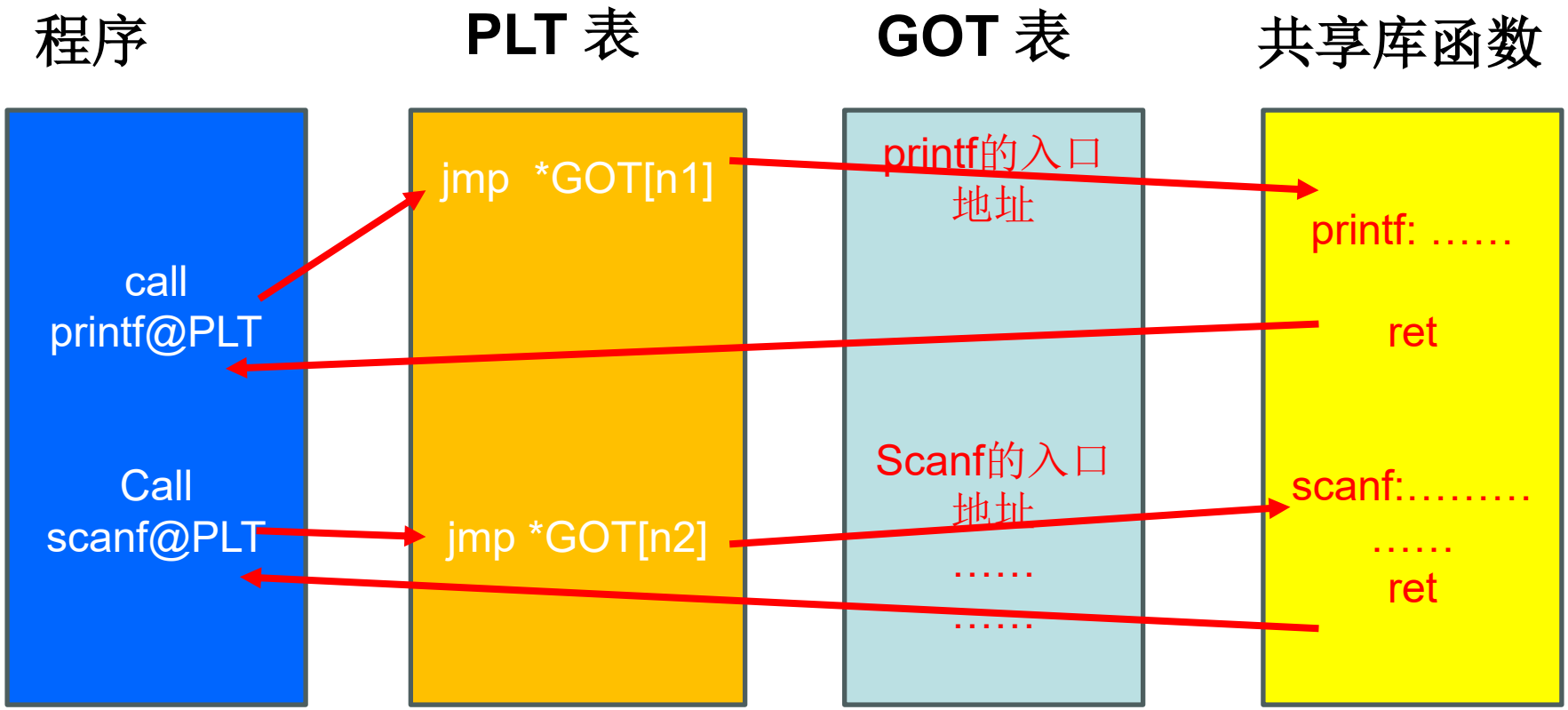
1）保证共享库代码的位置可以是不确定的

2）即使共享库代码的长度发生变化，也不会影响调用它的程序

gcc –c myproc1.c myproc2.c　　位置无关的共享代码库文件

gcc –shared –fPIC –o mylib.so myproc1.o myproc2.o

位置无关代码（PIC）



Procedure Linkage Table
过程链接表

Global Offset Table
全局偏移表

加载时，只需要填写**GOT表**， 如 第 **n1表项** 为 **printf的入口地址**

本章小结

- 链接处理涉及到三种目标文件格式：可重定位目标文件、可执行目标文件和共享目标文件。共享库文件是一种特殊的目标文件（PIC）。
- ELF目标文件格式有链接视图和执行视图两种，前者是可重定位目标格式，后者是可执行目标格式。
  - 链接视图中包含ELF头、各个节以及节头表
  - 执行视图中包含ELF头、程序头表（段头表）以及各种节组成的段
- 链接分为静态链接和动态链接两种
  - 静态链接将多个可重定位目标模块中相同类型的节合并起来，以生成完全链接的可执行目标文件，其中所有符号的引用都是在虚拟地址空间中确定的最终地址，因而可以直接被加载执行。
  - 动态链接的可执行目标文件是部分链接的，还有一部分符号的引用地址没有确定，需要利用共享库中定义的符号进行重定位，因而需要由动态链接器来加载共享库并重定位可执行文件中部分符号的引用。
    - 加载时进行共享库的动态链接
    - 执行时进行共享库的动态链接

本章小结

- 链接过程需要完成符号解析和重定位两方面的工作
  - 符号解析的目的就是将符号的引用与符号的定义关联起来
  - 重定位的目的是分别合并代码和数据，并根据代码和数据在虚拟地址空间中的位置，确定每个符号的最终存储地址，然后根据符号的确切地址来修改符号的引用处的地址。
- 在不同目标模块中不应定义相同符号，重定义。
- 不应该有未定义的符号。
- 加载器在加载可执行目标文件时，实际上只是把可执行目标文件中的只读代码段和可读写数据段通过页表映射到了虚拟地址空间中确定的位置，并没有真正把代码和数据从磁盘装入主存。

一些软件工具

#od -Ax -tx1 test.o // octal dump

od [-abcdfhilovx] [-A<字码基数>] [-j<字符数目>]
[-N<字符数目>] [-s<字符串字符数>] [-t<输出格式>]
[-w<每列字符数>] [--help] [--version] [文件名]

#hexdump -C -n 32 -s 0x40 test.o

从文件偏移 0x40 处，显示 32个字节

hexdump [-bcCdovx] [-e fmt] [-f fmt_file]
[-n length] [-s skip] [file ...]

```
root@LAPTOP-CJLSTBTI:/home/chapter4# hexdump -C  -n 512 test.o
00000000  7f 45 4c 46 02 01 01 00  00 00 00 00 00 00 00 00  |.ELF............|
00000010  01 00 3e 00 01 00 00 00  00 00 00 00 00 00 00 00  |..>.............|
00000020  00 00 00 00 00 00 00 00  68 05 00 00 00 00 00 00  |........h.......|
00000030  00 00 00 00 40 00 00 00  00 00 40 00 13 00 12 00  |....@.....@.....|
00000040  55 48 89 e5 89 7d ec 89  75 e8 8b 55 ec 8b 45 e8  |UH...}..u..U..E.|
00000050  01 d0 89 45 fc 8b 45 fc  5d c3 77 00 00 00 04 00  |...E..E.].w.....|
00000060  00 00 00 00 08 01 00 00  00 00 0c 00 00 00 00 00  |................|
```

pie

PIE： Position-Independent Executable

位置无关可执行文件

PIE 是专门针对普通可执行程序的编译属性，是 PIC 的衍生机制。它让整个可执行程序的所有指令、数据全部采用相对偏移寻址，不依赖固定的内存基地址，程序可以被操作系统加载到进程虚拟内存的任意地址。

GCC 8 及以下：可执行程序默认 非 PIE（非位置无关）

GCC 9 ~ GCC 15：默认编译出来的程序是 位置无关的

pie

```
xu@LAPTOP-CJLSTBTI:~/link_test$ gcc  -g -o linktest -D U1 -no-pie main.c  sub.c
xu@LAPTOP-CJLSTBTI:~/link_test$ readelf -s -W linktest | awk '$7==25'
    24: 0000000000404040     0 NOTYPE  WEAK   DEFAULT   25 data_start
    25: 0000000000404058    12 OBJECT  GLOBAL DEFAULT   25 init_gstr1
    27: 0000000000404094     0 NOTYPE  GLOBAL DEFAULT   25 _edata
    33: 0000000000404080     8 OBJECT  GLOBAL DEFAULT   25 init_gp2
    35: 0000000000404040     0 NOTYPE  GLOBAL DEFAULT   25 __data_start
    40: 0000000000404048     0 OBJECT  GLOBAL HIDDEN    25 __dso_handle
    42: 0000000000404050     4 OBJECT  GLOBAL DEFAULT   25 init_gv1
    47: 0000000000404088     8 OBJECT  GLOBAL DEFAULT   25 init_gfp1
    51: 0000000000404074     4 OBJECT  GLOBAL DEFAULT   25 gv
    52: 0000000000404078     8 OBJECT  GLOBAL DEFAULT   25 init_gp1
    53: 0000000000404068    12 OBJECT  GLOBAL DEFAULT   25 init_garr1
    54: 0000000000404090     4 OBJECT  GLOBAL DEFAULT   25 extern_int
```

```
(gdb) p &init_gv1
$1 = (int *) 0x404050 <init_gv1>
(gdb)
```

pie

- x86_64 下，NO-PIE、PIE ，对于全局变量的访问，全部使用 RIP 相对寻址，无绝对地址指令；

NO-PIE 链接固化固定基址（默认固定基址 0x400000）

提前计算好每一个全局变量的最终绝对虚拟地址。

程序加载时：操作系统必须强制加载到 0x400000，不允许更改基址。

readelf 读取的链接固化绝对地址 = GDB 运行加载后的内存地址

因为基地址写死，NO-PIE 彻底不支持 ASLR，无法地址随机化。

- PIE 不固化基址，依赖系统随机基址，地址可变，readelf 与 GDB 地址不一致，支持 ASLR。

结语

- 编译、链接技术在不断进步
- 相同的程序，用某个编译器可生成执行程序，但换一个编译器有可能编译失败，不能生成执行程序
- 不同编译开关，生成的执行程序并不相同
- 链接的基本概念并不复杂

  符号（全局变量，非静态局部变量，函数）

  符号解析：符号的定义 与 符号的使用关联起来

  符号重定位：修正代码中“临时偏移/未知地址”，把代码里悬空的地址引用，替换成真实可用的内存地址的过程。
- 对于操作系统而言，加载程序时，地址映射非常复杂，包含内存映射、页表构建、ASLR随机、动态重定位、库加载等